



Metaheuristic Optimization



**Heuristic Algorithms in Transport
and Finance**

Optimization Modeling 101

Prof. Dr. Angel A. Juan

Full Professor, Applied Stats and Operations Research Dept.

ajuanp@upv.es

ICSO
Analytics



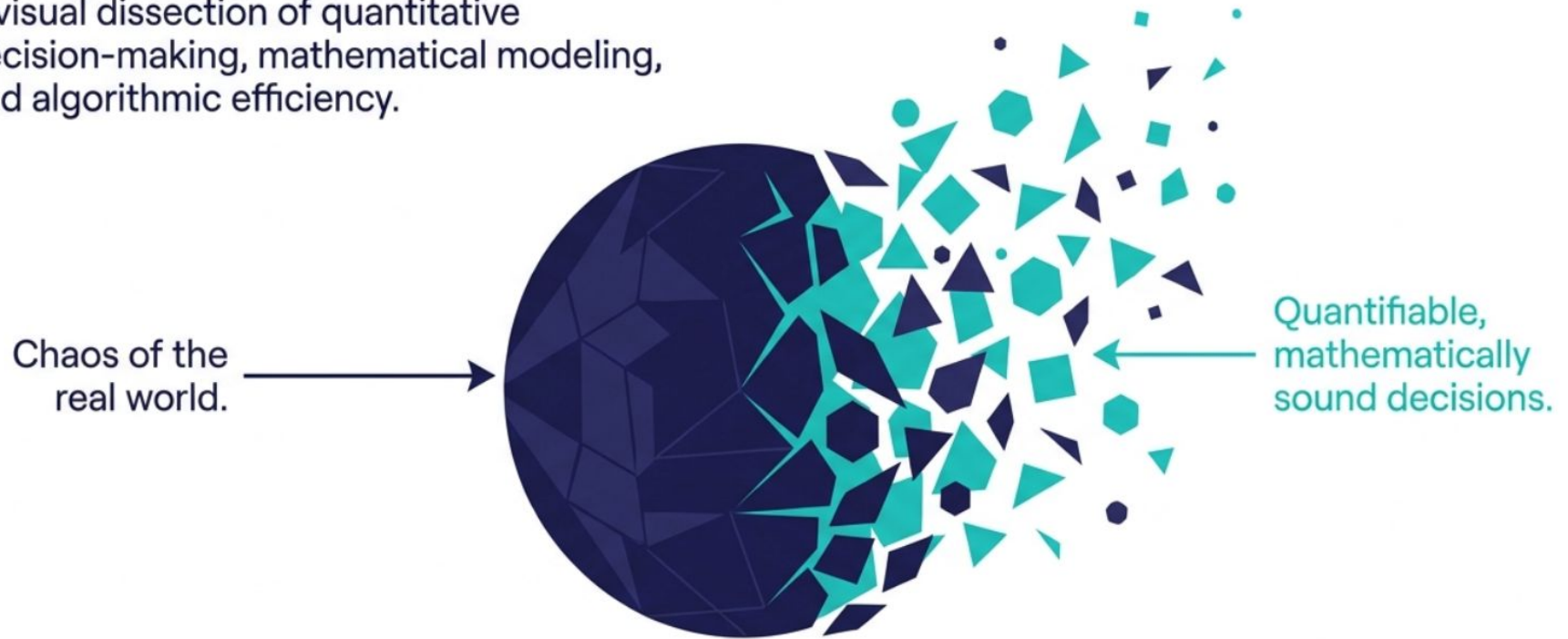
UNIVERSITAT
POLITÈCNICA
DE VALÈNCIA



valgrAI
Valencian Graduate School
and Research Network
of Artificial Intelligence

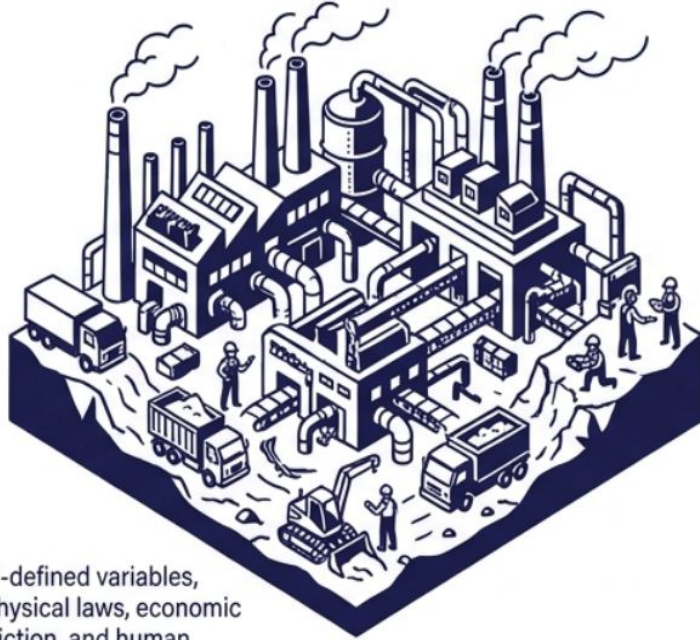
Operations Research: The Decision Blueprint

A visual dissection of quantitative decision-making, mathematical modeling, and algorithmic efficiency.



Translating Reality into Mathematics

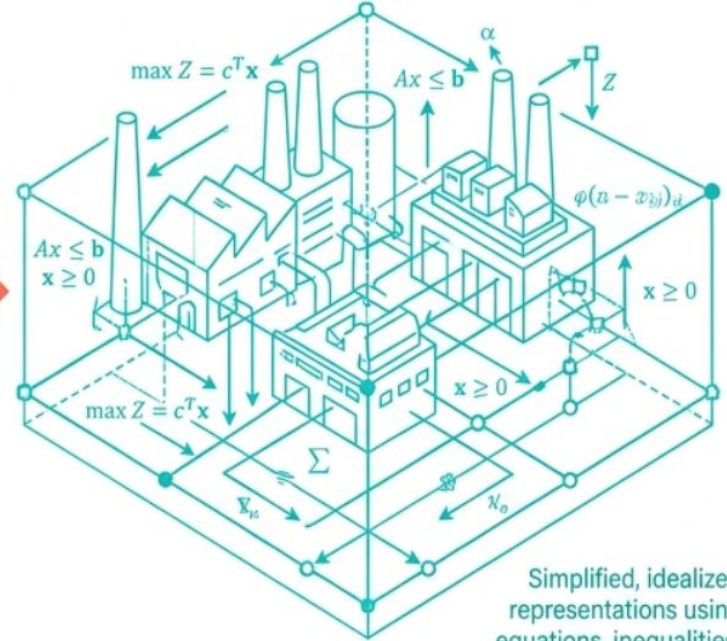
Physical Reality



Ill-defined variables,
physical laws, economic
friction, and human
motivations

The Translation Process

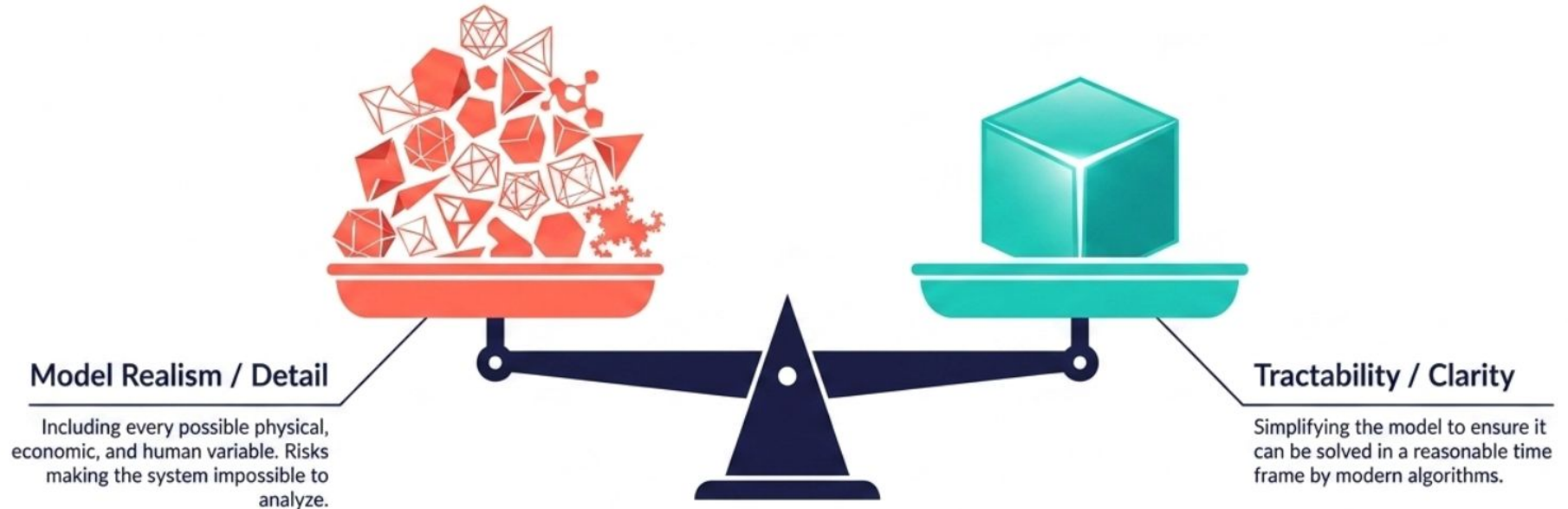
Mathematical Model



Simplified, idealized
representations using
equations, inequalities,
and objective functions

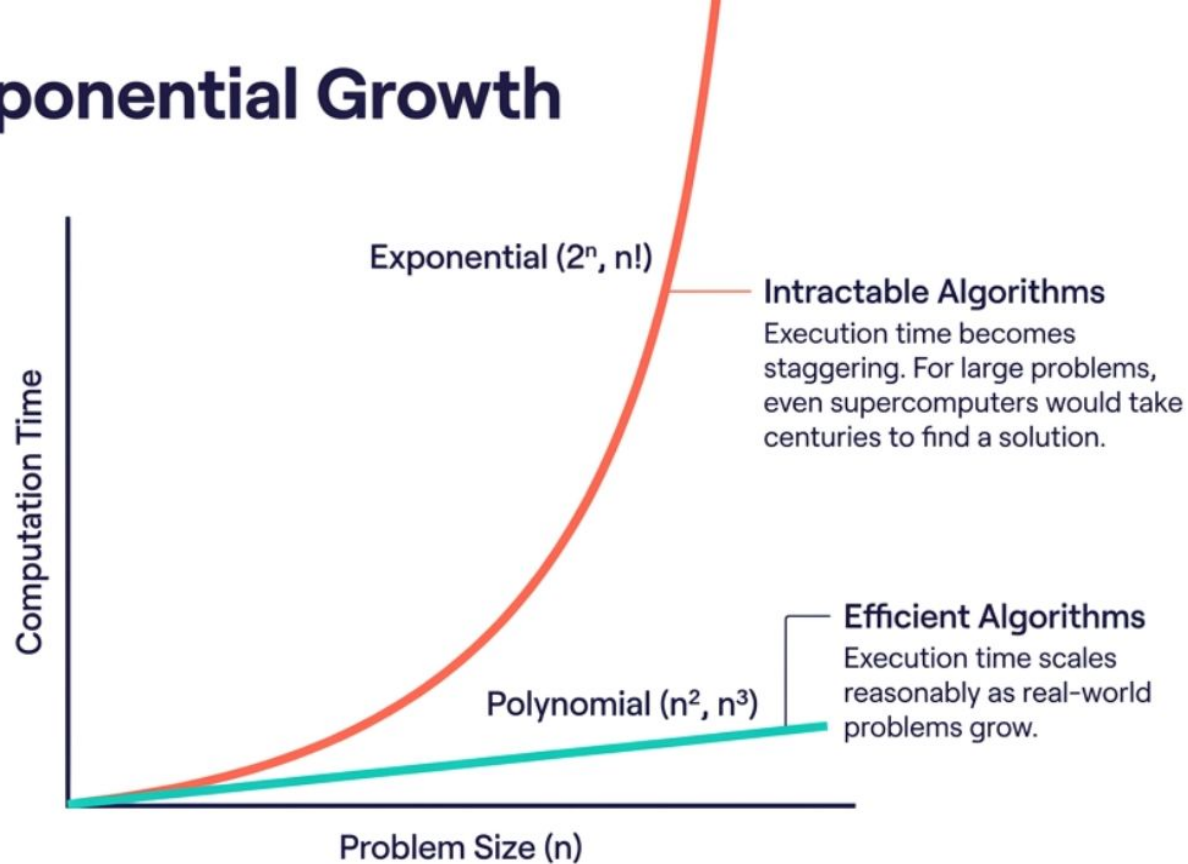
Key Takeaway: Operations Research uses quantitative methods to help decision-makers optimize systems—turning real-world chaos into actionable, mathematically verified strategies.

The Tractability Trade-Off



It is tempting to confuse detail with accuracy. Not all details are correct, and not all details are relevant. The best model strikes a practical compromise.

The Trap of Exponential Growth



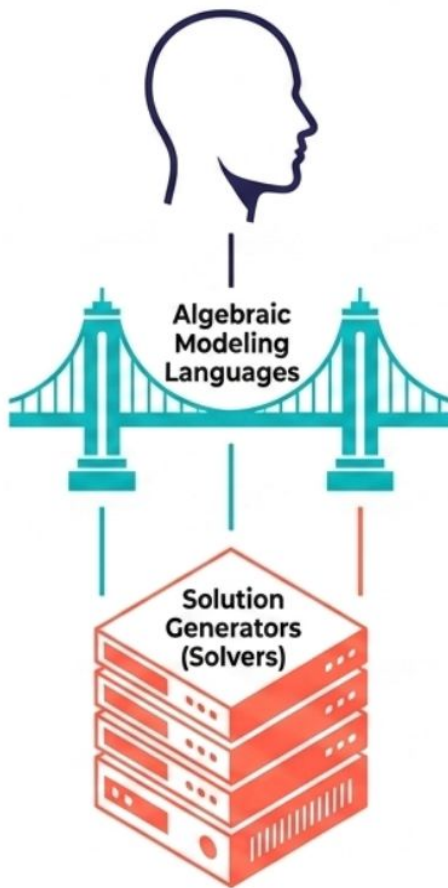
**An algorithm can be mathematically correct
but practically useless.**

Diagnosing Algorithm Viability

Algorithm Type	Big-Oh Notation	Time on Small Inputs	Time on Large Inputs	Real-World Viability
Constant	$O(1)$	Instant	Instant	Maximum
Linear / Polynomial	$O(n)$, $O(n^2)$	Fast	Manageable	Highly Practical
Exponential	$O(2^n)$	Fast	Centuries	Severely Limited
Factorial	$O(n!)$	Fast	Staggering	Theoretical Only

Big-Oh notation describes the worst-case performance of an algorithm as problem size (n) increases.

Bridging the Gap: OR Software



The Human Element

The analyst formulates the logical problem and gathers raw data.

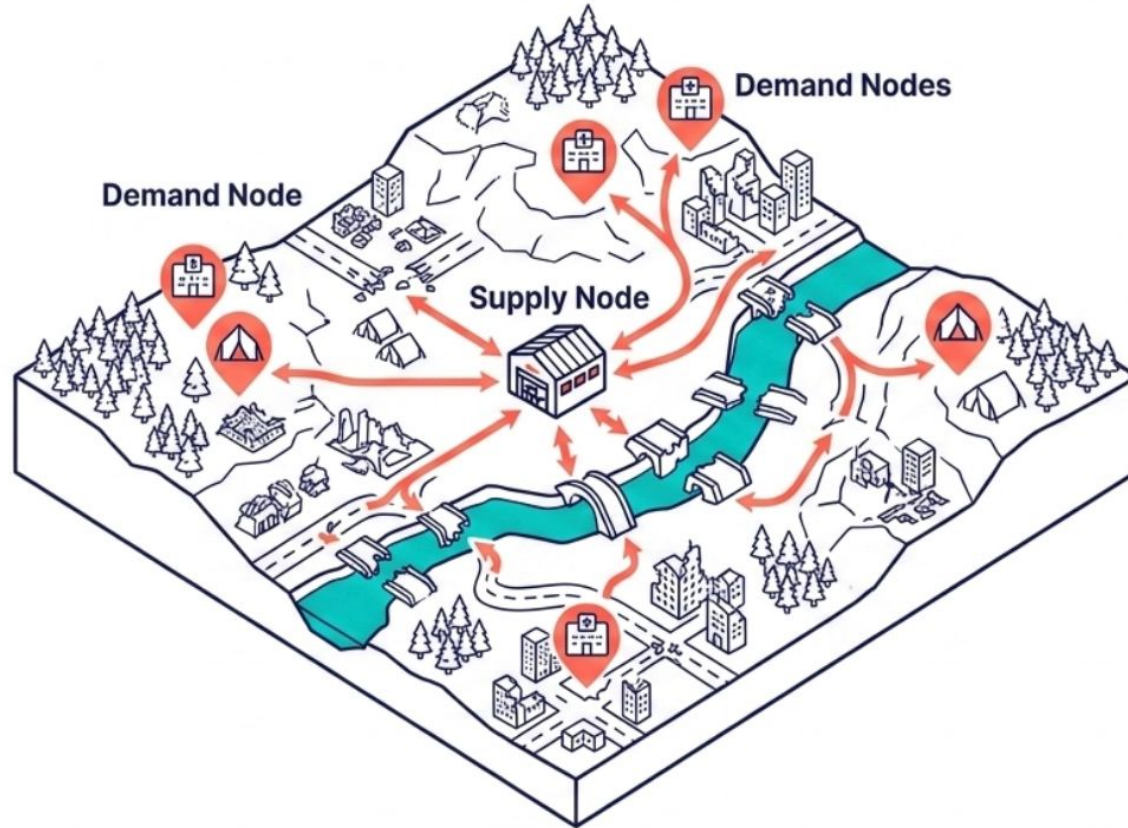
Modeling Environments

Software systems that translate human logic into mathematical representations, preparing parameters for calculation.

The Solvers

High-powered, specialized computational software that applies the algorithms to find the optimal solution.

Application in Crisis: Humanitarian Relief



The Problem

Distributing scarce survival supplies in the immediate wake of a natural disaster.

The Constraints

Destroyed infrastructure, limited vehicles, unpredictable weather, and extreme time sensitivity.

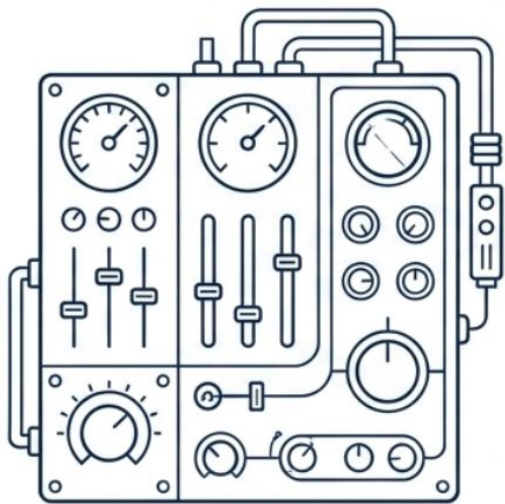
The OR Solution

Using network models and algorithmic routing to optimize supply chains, ensuring maximum possible relief delivery despite chaotic ground conditions.

Part I:

Optimization Basics

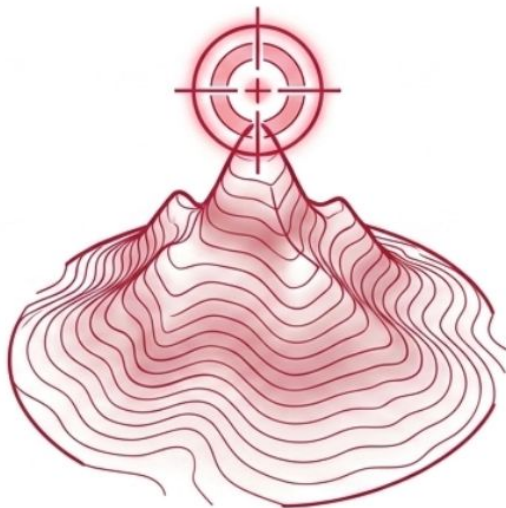
The Universal Anatomy of an Optimization Problem



Decision Variables:

$$\mathbf{x} = (x_1, \dots, x_n)$$

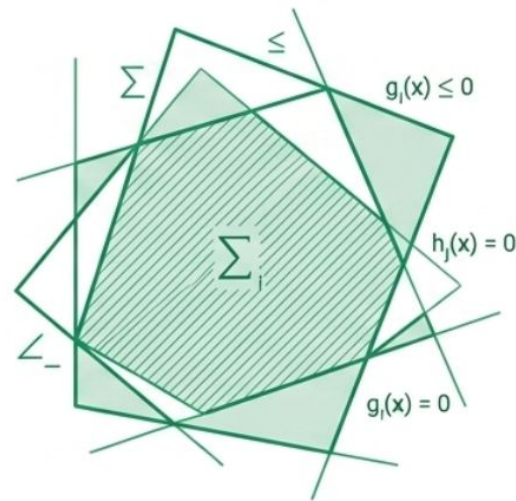
The choices we can actively control or change.



Objective Function:

$$f(\mathbf{x})$$

The goal we want to maximize (e.g., profit) or minimize (e.g., cost).



Constraints:

$$g_i(\mathbf{x}) \leq 0 \quad h_j(\mathbf{x}) = 0$$

The strict rules, budgets, and physical limits bounding our choices. Only some solutions are physically feasible.

Formulating Optimization Problems

- The formulation of an **optimization problem** consists of three parts:
 - A set of **decision variables** $\mathbf{x} = (x_1, x_2, \dots, x_n)$.
 - An **objective function** to maximize or minimize, $f = f(\mathbf{x})$.
 - A set of **constraints**, $g_i(\mathbf{x}) \leq 0$ (inequality constraints) and $h_j(\mathbf{x}) = 0$ (equality constraints), with i in I , j in J .
- An **unconstrained** optimization problem is typically solved with Calculus techniques.
- When there are constraints, only some solutions $\mathbf{x}$ are **feasible**.
- It is possible to have **multiple optimal solutions**.

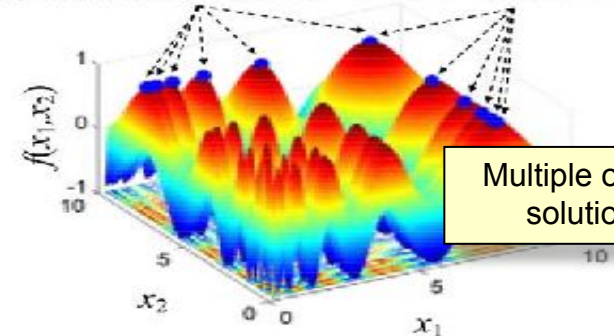
Optimization problem formulation.

$$\begin{array}{ll} \text{minimize} & f(\mathbf{x}) \\ \text{such that} & h_i(\mathbf{x}) = 0, \quad i = 1, \dots, n_e \\ & g_j(\mathbf{x}) \leq 0, \quad j = 1, \dots, n_g \end{array}$$

Calculus techniques for unconstrained optimization.

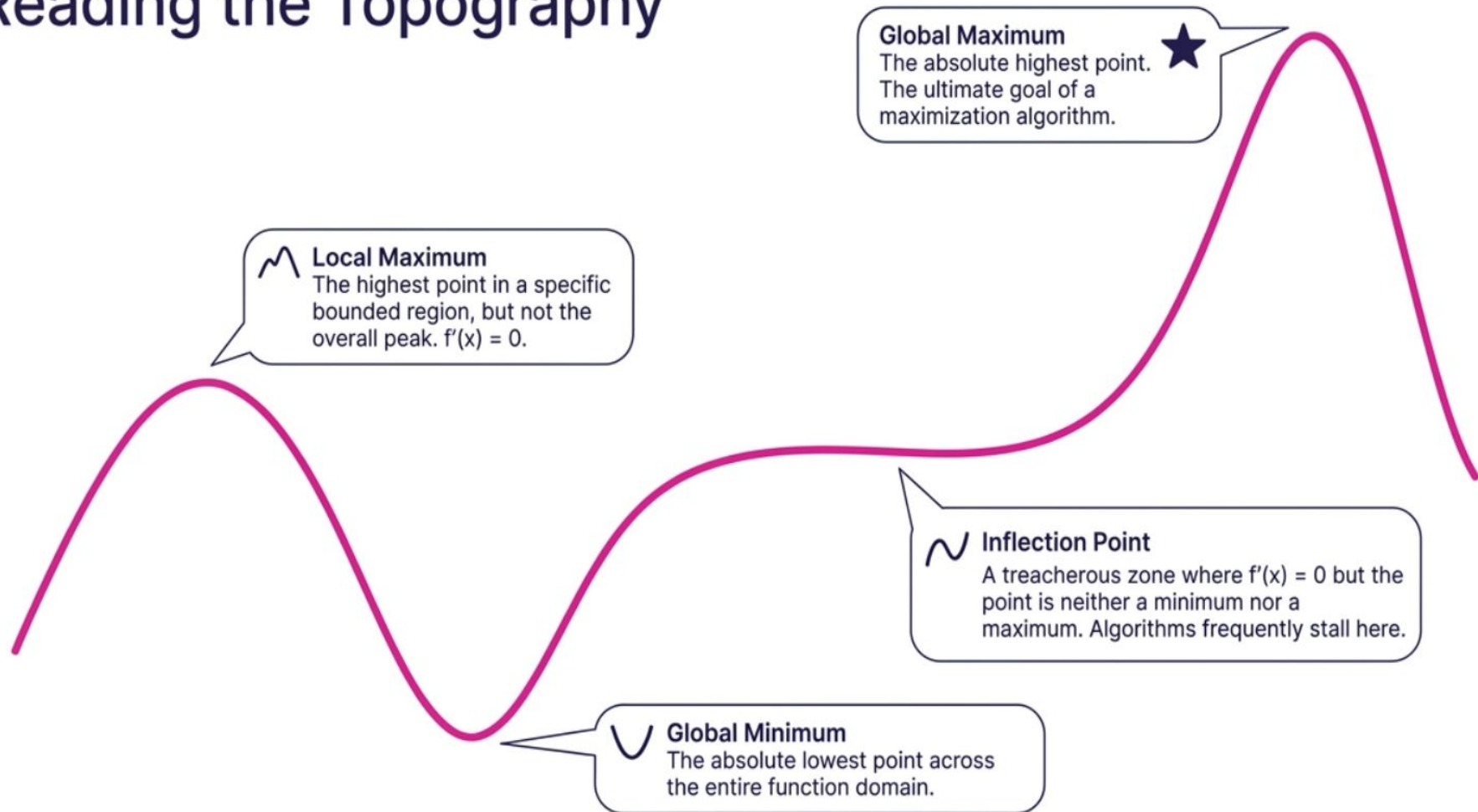
$$f_{x_1} = \frac{\partial f(x_1^*, x_2^*)}{\partial x_1} = 0$$
$$f_{x_2} = \frac{\partial f(x_1^*, x_2^*)}{\partial x_2} = 0$$

Some optimal solutions have the same value in the second decision variable Some optimal solutions have the same value in the first decision variable



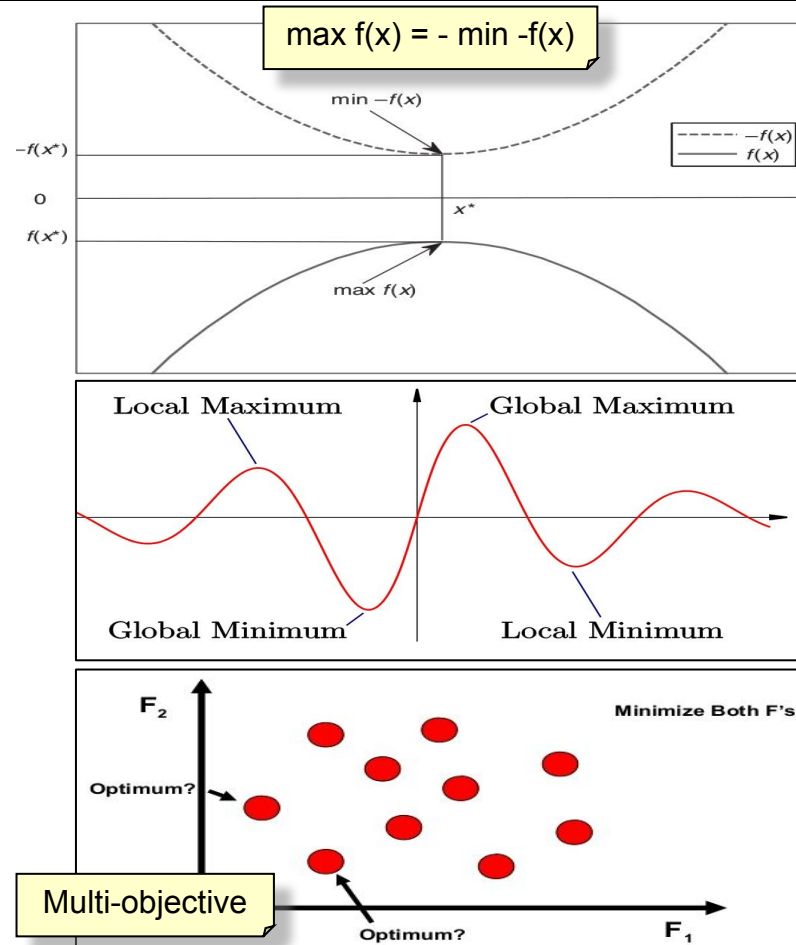
Multiple optimal solutions.

Reading the Topography



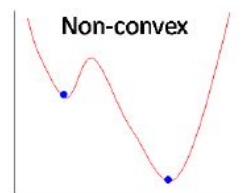
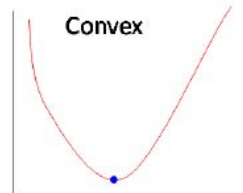
Basic Concepts

- Finding the **maximum** of a function $f(x)$ is equivalent to finding the **minimum** of $-f(x)$ and then change its sign, and vice versa, i.e.: $\max f(x) = -\min -f(x)$ and $\min f(x) = -\max -f(x)$.
- A **global optimal** solution is the best one for any feasible value of x . A **local optimal** solution is the best one in a neighborhood of feasible solutions.
- Under some **convexity** conditions, a local optimal solution is guaranteed to be the global optimal.
- A **multi-objective opt. problem** can be reformulated as a single-objective one by either:
(a) assigning **weights** to the different objectives;
or (b) optimizing the most important one and include the others as **constraints**.



Convex Optimization

- **Convex programming:**
 - Both $f(x)$ and $g_i(x)$ are convex functions, and $Ax = b$ is a system of linear equalities.
 - Includes LP, and some types of QP, SOCP, and semidefinite programming.
 - Local optimal solutions are also global.
- **Linear programming (LP):** both $f(x)$ and the constraints are linear expressions in the decision variables.
- **Quadratic programming (QP):** $f(x)$ is a quadratic expression in the decision variables, with Q a positive semidefinite matrix.
- **Second-order cone programming (SOCP):** $\|\cdot\|$ stands for second (Euclidean) norm.
- **Integer programming (IP) and mixed integer programming (MIP):** all decision variables are integer (IP) or some of them are integer (discrete) and some continuous (MIP).



$$\begin{array}{ll}\min_x & f(x) \\ \text{subject to} & g_i(x) \leq 0 \quad i = 1, \dots, I \\ & Ax = b\end{array}$$

$$\begin{array}{ll}\min_x & c'x \\ \text{subject to} & Ax = b \\ & x \geq 0\end{array}$$

Linear programming

Quadratic programming

$$\begin{array}{ll}\min_x & \frac{1}{2}x'Qx + c'x \\ \text{subject to} & Ax = b \\ & x \geq 0\end{array}$$

$$\begin{array}{ll}\min_x & c'x \\ \text{subject to} & Ax = b \\ & \|C_i x + d_i\| \leq c'_i x + e_i, \quad i = 1, \dots, I\end{array}$$

Integer programming

and $x \in \mathbb{Z}^n$

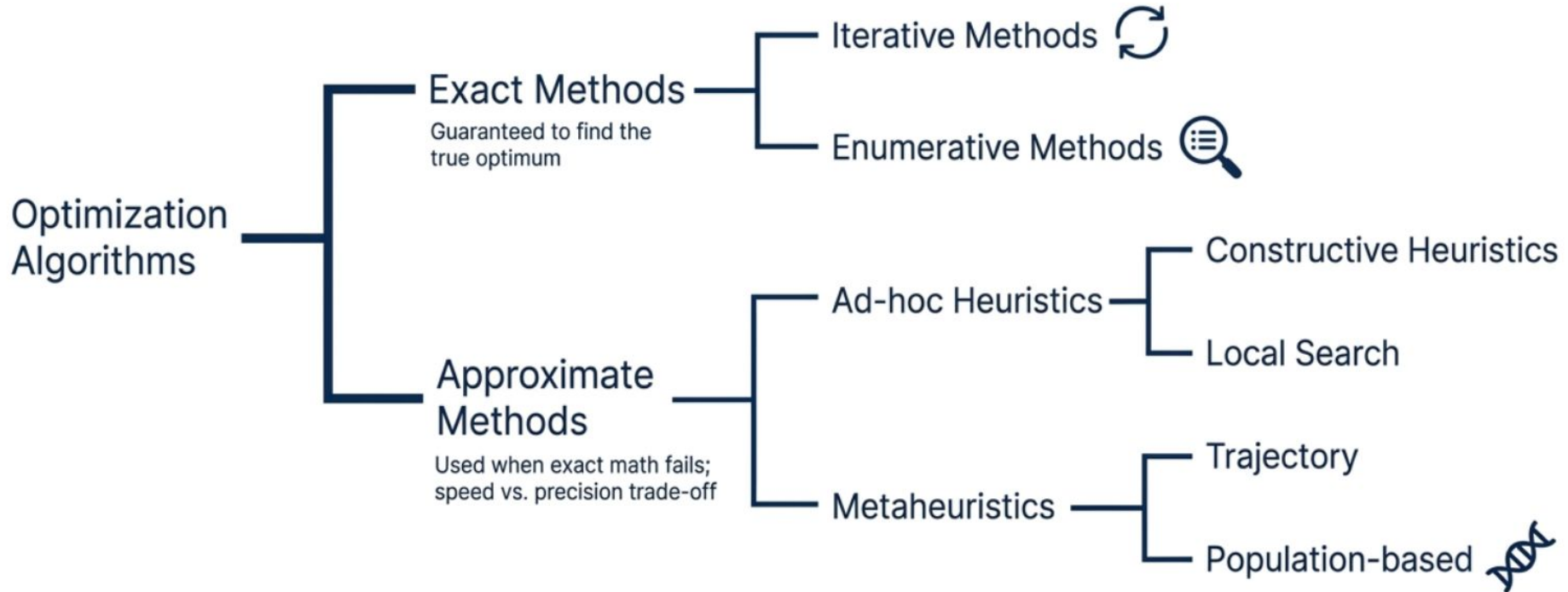
A Taxonomy of Convex Optimization

Under convexity conditions, any local optimal solution is guaranteed to be the global optimal solution.

Type	Objective $f(x)$	Constraints	Visual Signature
Linear Programming (LP)	Linear expressions	Linear expressions	
Quadratic Programming (QP)	Quadratic expression (with positive semidefinite matrix Q)	Linear expressions	
Second-Order Cone (SOCP)	Linear expressions	2 nd (Euclidean) norm limits	
Integer/Mixed (IP/MIP)	Linear expressions	Variables restricted to discrete integers	

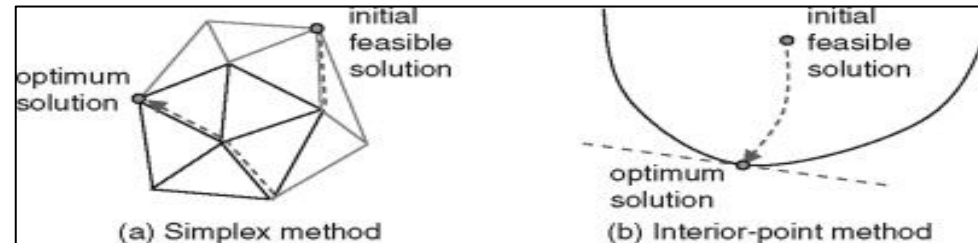
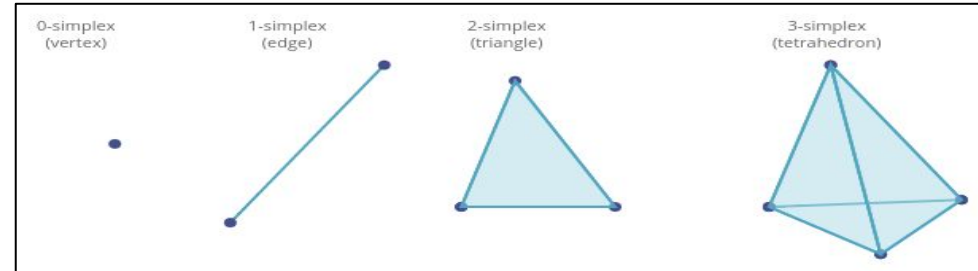
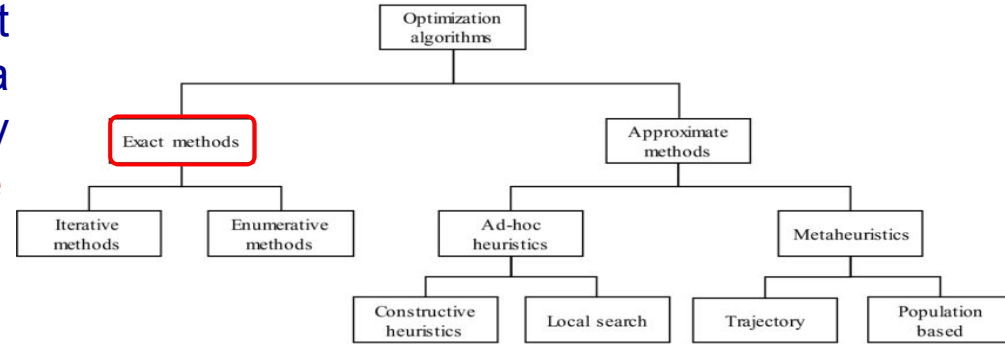
The Algorithmic Toolkit

We deploy different algorithmic families depending on the complexity of the mathematical terrain.



Optimization Algorithms (1/3)

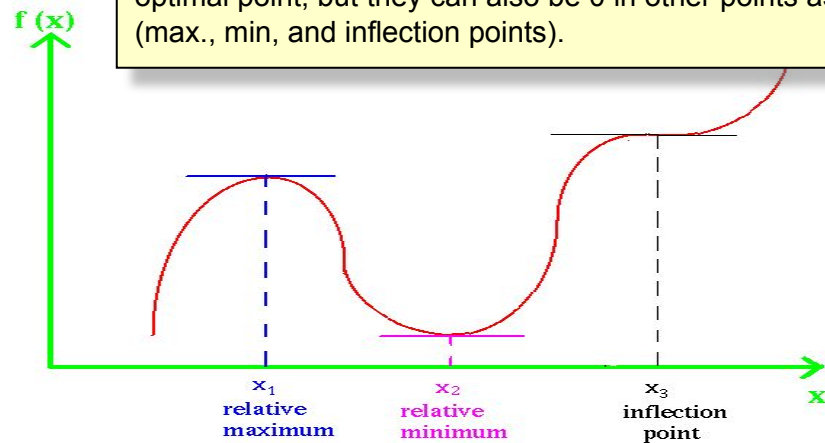
- Most opt. algorithms are iterative: start with an initial solution and generate a sequence of intermediate sols until they get “close”, according to a **tolerance** parameter, to the optimal one.
- Linear** optimization algorithms:
 - The **simplex algorithm** (Dantzig, 1947) solves LPs by iteratively solving systems of linear equations. It traverses the **simplex** of feasible points along the edges and only considers the corners.
 - Interior point methods** (Karmakar, 1980s) reach the opt. solution from inside of the feasible set. They can be applied to a wider class of convex opt. problems.



Optimization Algorithms (2/3)

- Constrained nonlinear optim.:
 - Many nonlinear opt. algorithms are based on the **Kuhn-Tucker (KT) necessary conditions** for optimality.
 - In unconstrained optimization, the **derivative** of $f(x)$ in an optimal point x^* equals 0 (necessary but not sufficient condition).
 - KT conditions are based on principles of Calculus: (a) transform the constrained problem into an unconstrained one by using the **Lagrangian** form; and (b) consider a set of **necessary equations** forcing derivatives to be 0.
 - KT conditions enable a nonlinear opt. problem to be reduced to a system of nonlinear equations.

If $f(x)$ is differentiable, its derivatives are always 0 in a local optimal point, but they can also be 0 in other points as well (max., min, and inflection points).



Combining Lagrange conditions for equality and inequality constraints yields **KKT conditions** for general problem:

$$\min_x f(x)$$

$$\text{s. t. } \begin{aligned} \mathbf{g}(\mathbf{x}) &\leq \mathbf{0} \\ \mathbf{h}(\mathbf{x}) &= \mathbf{0} \end{aligned}$$

Lagrangian:

$$L = f(\mathbf{x}) + \boldsymbol{\mu}^T \mathbf{g}(\mathbf{x}) + \boldsymbol{\lambda}^T \mathbf{h}(\mathbf{x})$$

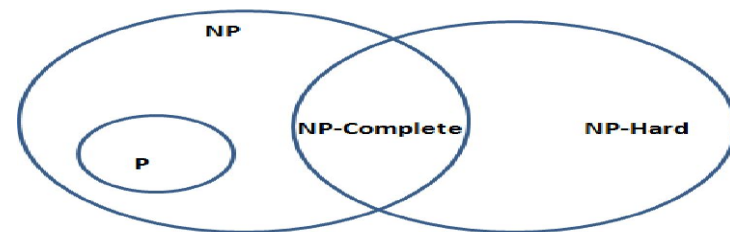
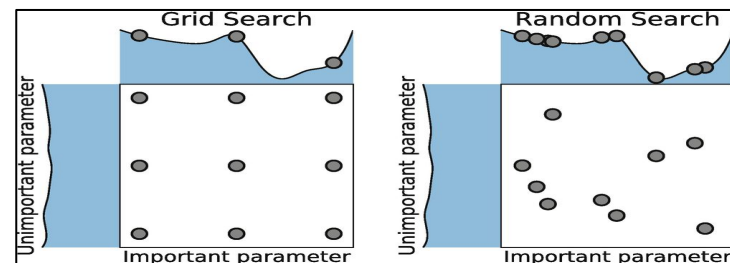
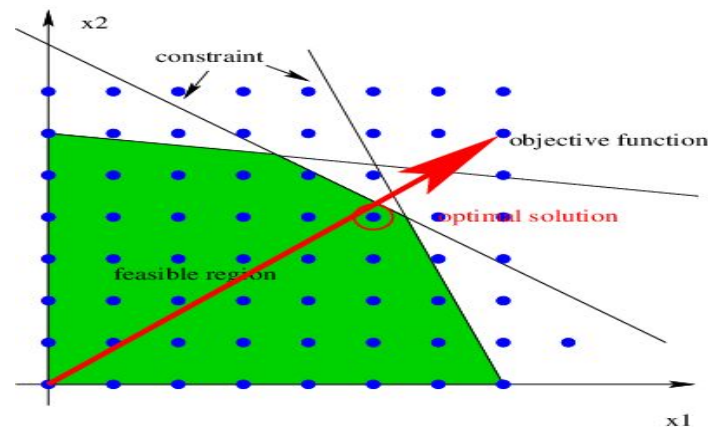
$$\Rightarrow \frac{\partial L}{\partial \mathbf{x}} = \frac{\partial f}{\partial \mathbf{x}} + \sum \mu_i \frac{\partial g_i}{\partial \mathbf{x}} + \sum \lambda_i \frac{\partial h_i}{\partial \mathbf{x}} = \mathbf{0} \quad (\text{optimality})$$

$$\text{and } \mathbf{g} \leq \mathbf{0}, \quad \mathbf{h} = \mathbf{0} \quad (\text{feasibility})$$

$$\boldsymbol{\lambda} \neq \mathbf{0}, \quad \boldsymbol{\mu} \geq \mathbf{0}, \quad \mu_i g_i = 0 \quad (\text{complementarity})$$

Optimization Algorithms (3/3)

- Integer programming algorithms:
 - Integer (IP) and mixed integer (MIP) optimization problems are generally more difficult to solve than LPs.
 - Simply rounding the LP optimal solution might lead to suboptimal solutions.
 - IPs and MIPs are typically solved by branch-and-bound algorithms, branch-and-cut routines (typically by first solving a LP version of the problem and then reducing the choices of integer solutions), and heuristics.
- Randomized search algorithms:
 - Incorporate an element of randomness in their search.
 - Simulated annealing, tabu search, genetic algorithms, etc.
 - Might be the only way to solve large-scale NP-hard (most difficult) optimization problems.
 - Do not guarantee optimality.



Optimization Methods

Optimization methods

Exact methods

Approximate methods

Branch and X

Constraint
programming

Dynamic
programming

A*, IDA*

Heuristic algorithms

Approximation
algorithms

Branch and
bound

Branch and
cut

Branch and
price

Metaheuristics

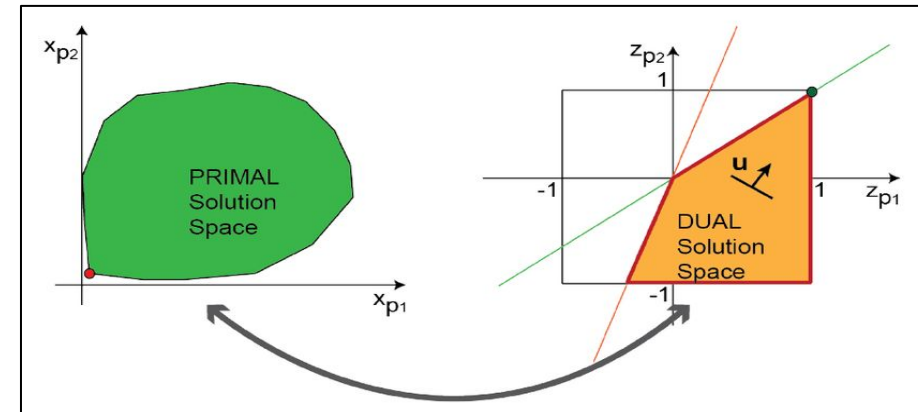
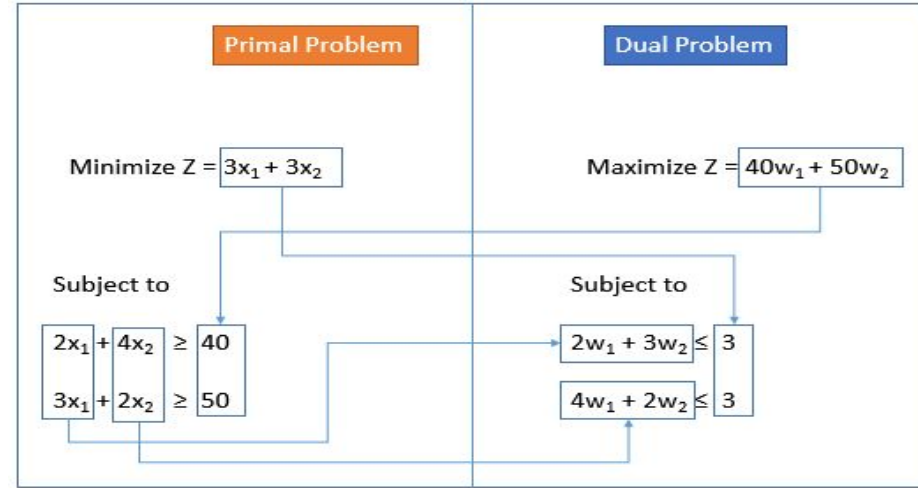
Problem-specific
heuristics

Single-solution based
metaheuristics

Population-based
metaheuristics

Optimization Duality

- Any **primal** (P) optimization problem has its **dual** (D) counterpart:
 - If P is a MIN problem, D is a MAX problem, and vice versa.
 - The **number of variables** in D is equal to the **number of constraints** in P, and vice versa.
 - A good solution to D can be used to compute a **bound** for the value of the objective function in P.
 - Often, D has better mathematical or computational **structure** than P.
 - Dual variables can be used to perform **sensitivity analysis** on P.



The Peril of Multi-Period Decisions

When decisions dictate strategies over multiple time periods, state space is represented by decision trees.

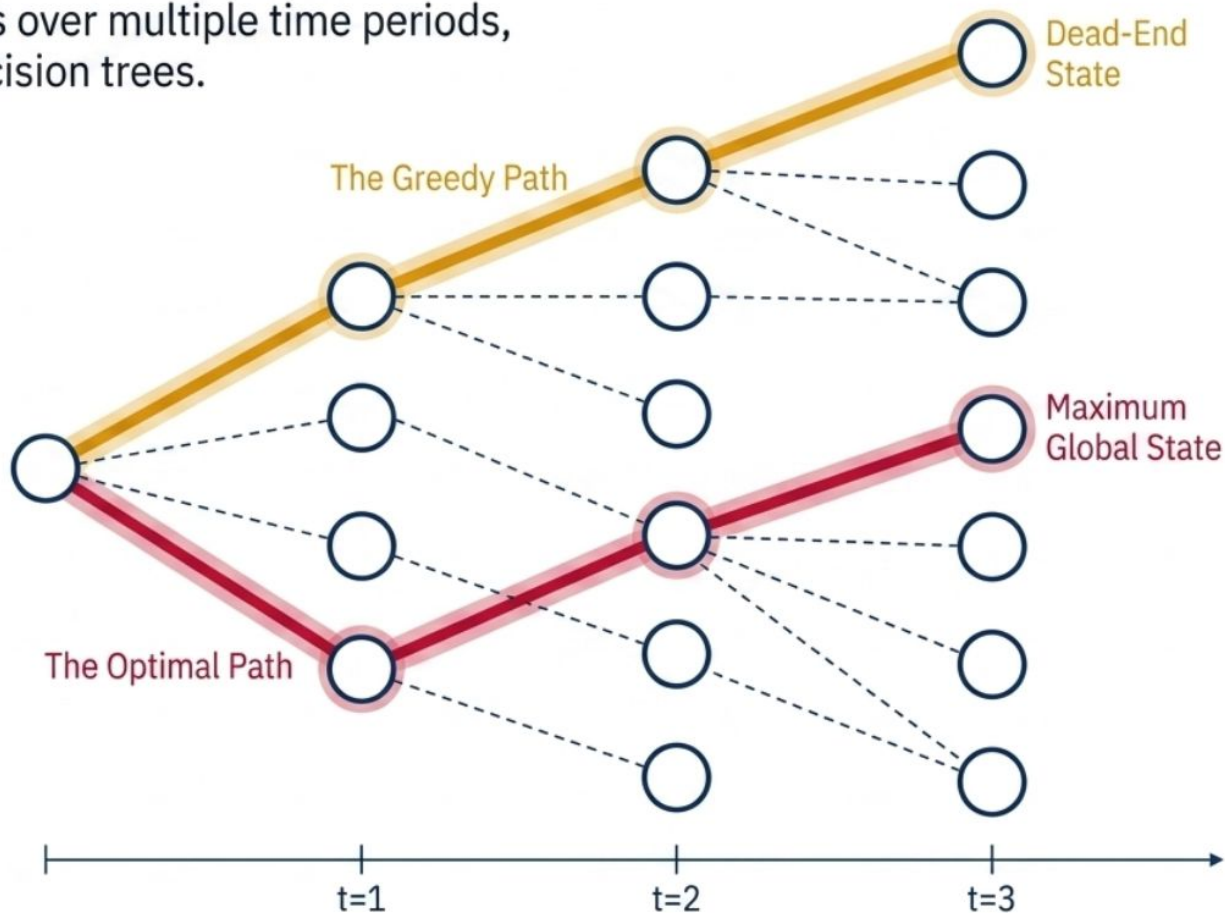
The Greedy Trap

Choosing the best option at each immediate step (greedy behavior) without looking ahead leads to suboptimal results.

Why? Because each decision changes the state and permanently limits what is possible later.

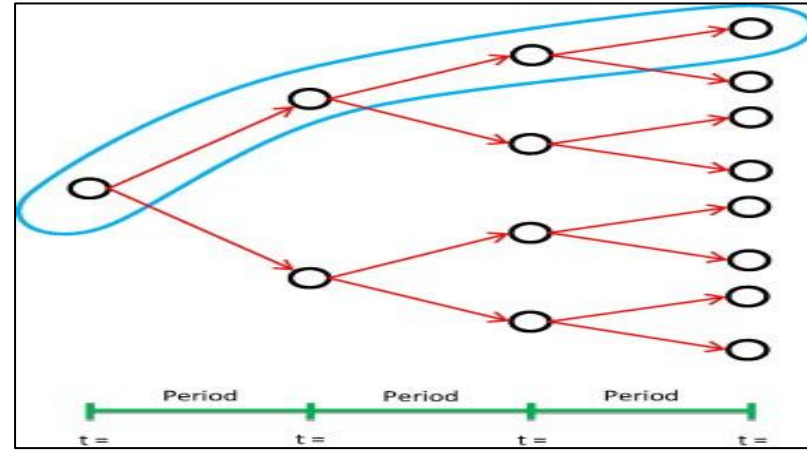
The Principle of Optimality

A necessary condition for an optimal decision is that it must lead to a state from which a globally optimal path can still be achieved.

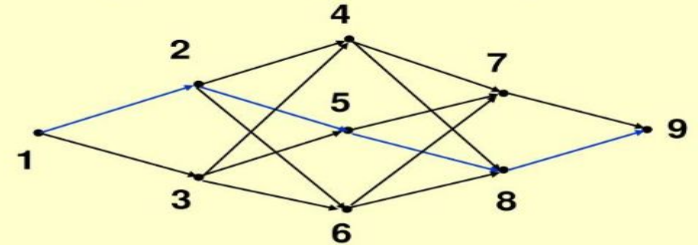


Multi-Period Optimization

- Some decisions involve finding optimal strategies over **multiple time periods** ahead.
- Typically, **decision trees** are employed to represent the **state space**.
- In a multi-period problem, **greedy behavior** (choosing the best option at each step without looking ahead) can lead to suboptimal results because each decision **changes the state and limits** what is possible later.
- Principle of optimality**: a necessary (though not sufficient) condition for a decision to be part of the optimal solution is that it leads to a state from which a globally optimal path can still be achieved.



Principle of Optimality



Bellman's Principle of Optimality: *At any intermediate state x_i in an optimal path from x_0 to x_f , the policy from x_i to goal x_f must itself constitute optimal policy*

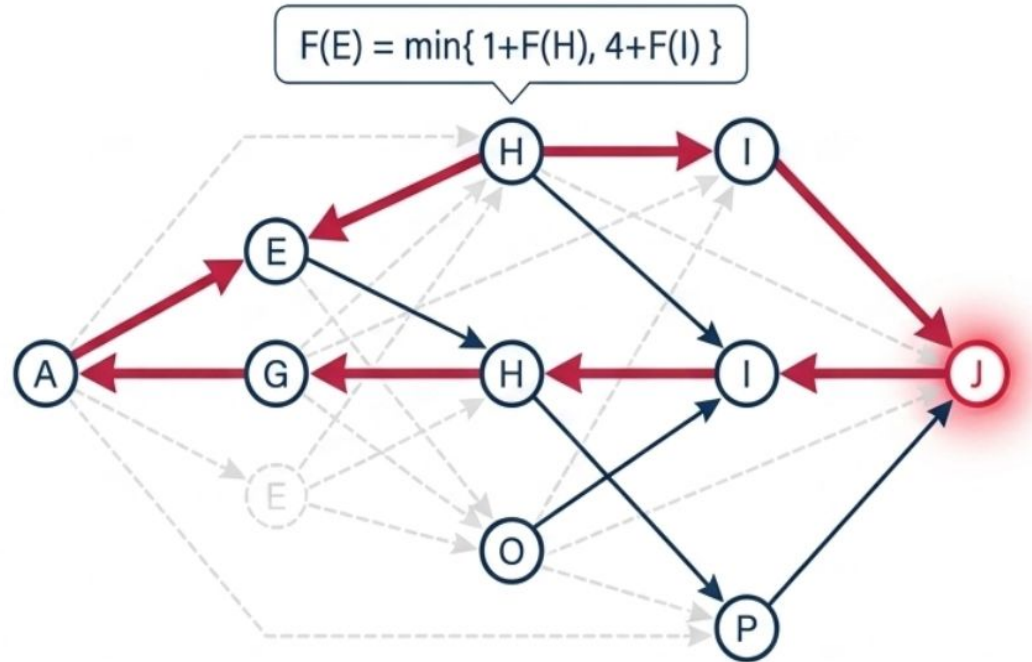
Dynamic Programming: Backward Induction

To solve a multi-period problem, we work backwards using a recursive technique.

Bellman's Principle of Optimality: An overall optimal solution must inherently contain optimal sub-paths.

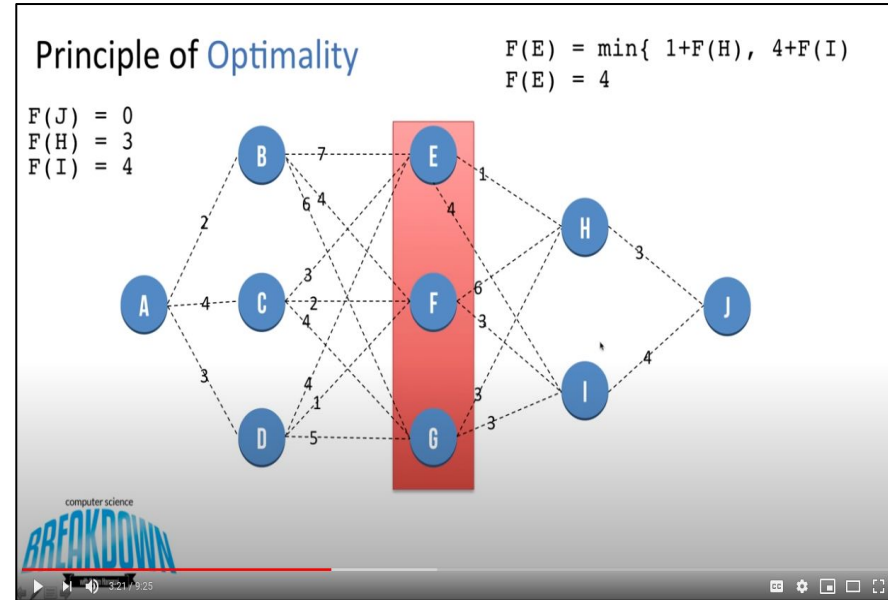
The Mechanic: We start from the final nodes at the end of the horizon where the best outcome is known. We record only the value of the best continuation from that state, working leftward step-by-step.

Efficiency: Suboptimal continuations are immediately discarded, massively reducing the computational search space.



Dynamic Programming

- Dynamic programming: to solve a multi-period problem we work **backwards** using a **recursive** technique, **starting from the final nodes**. At the end of the horizon the best outcome is known, and this information is used step by step to determine the best choices earlier on.
- Since an overall optimal solution must contain optimal sub-paths, we only need to record, at each node, **the value of the best continuation from that state**. Suboptimal continuations can be ignored because, according to the **principle of optimality**, they cannot be part of any global optimum.

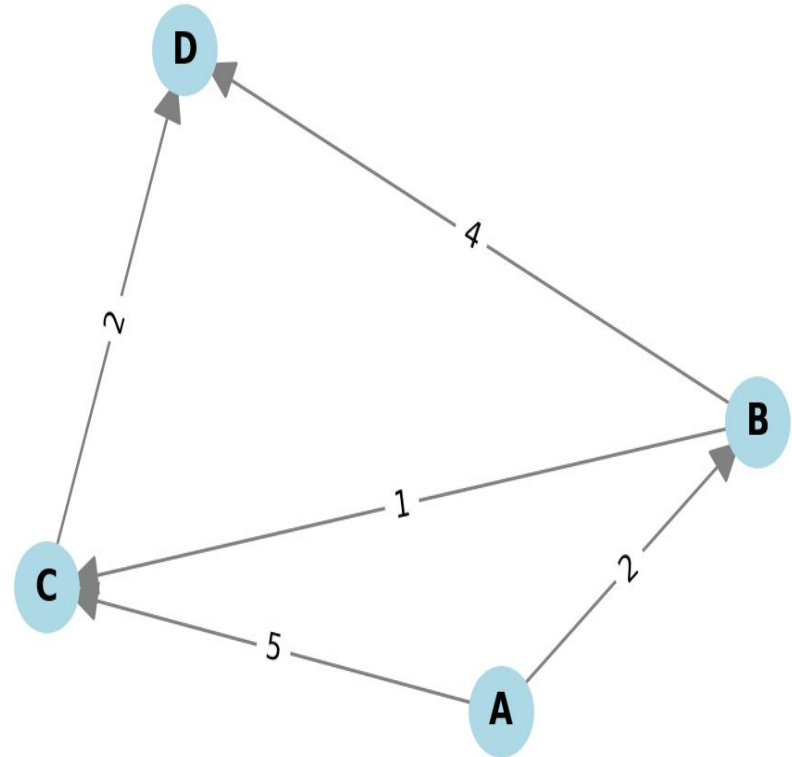


https://www.youtube.com/watch?v=_zE5z-KZGRw

Dynamic Programming Example

- Suppose you want the **shortest path** from **A to D** in this directed, weighted graph: A $\rightarrow$ B (cost 2), A $\rightarrow$ C (cost 5), B $\rightarrow$ C (cost 1), B $\rightarrow$ D (cost 4), C $\rightarrow$ D (cost 2).
- Each **node is a state**. The problem is to find the shortest cost from each state to the destination D.
- At the final **node D**, the cost to reach D is 0.
- From C**, the only choice is C $\rightarrow$ D with cost 2, so $\text{shortest}(C) = 2$.
- From B**, we have two choices: B $\rightarrow$ C $\rightarrow$ D ($1 + 2 = 3$) or B $\rightarrow$ D directly (4). The minimum is 3, so $\text{shortest}(B) = 3$.
- From A**, we have two choices: A $\rightarrow$ B $\rightarrow$... ($2 + 3 = 5$) or A $\rightarrow$ C $\rightarrow$... ($5 + 2 = 7$). The minimum is 5, so $\text{shortest}(A) = 5$.

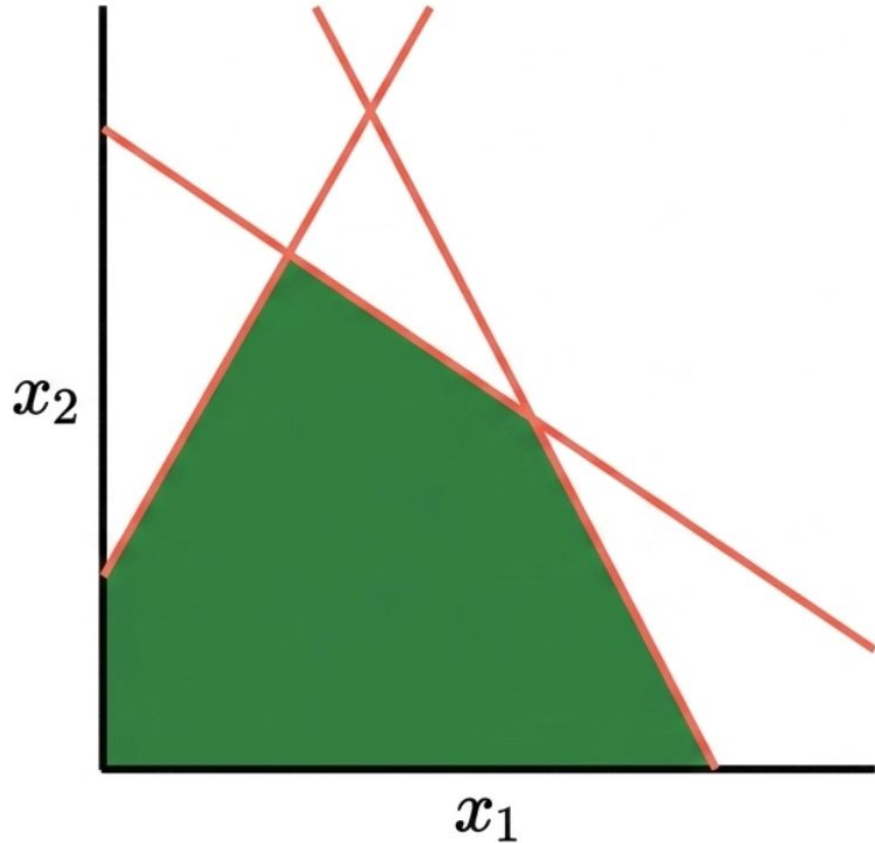
Shortest Path Example Graph



Part II:

Solving Linear Programming Problems

Visualizing the Problem: The Feasible Region



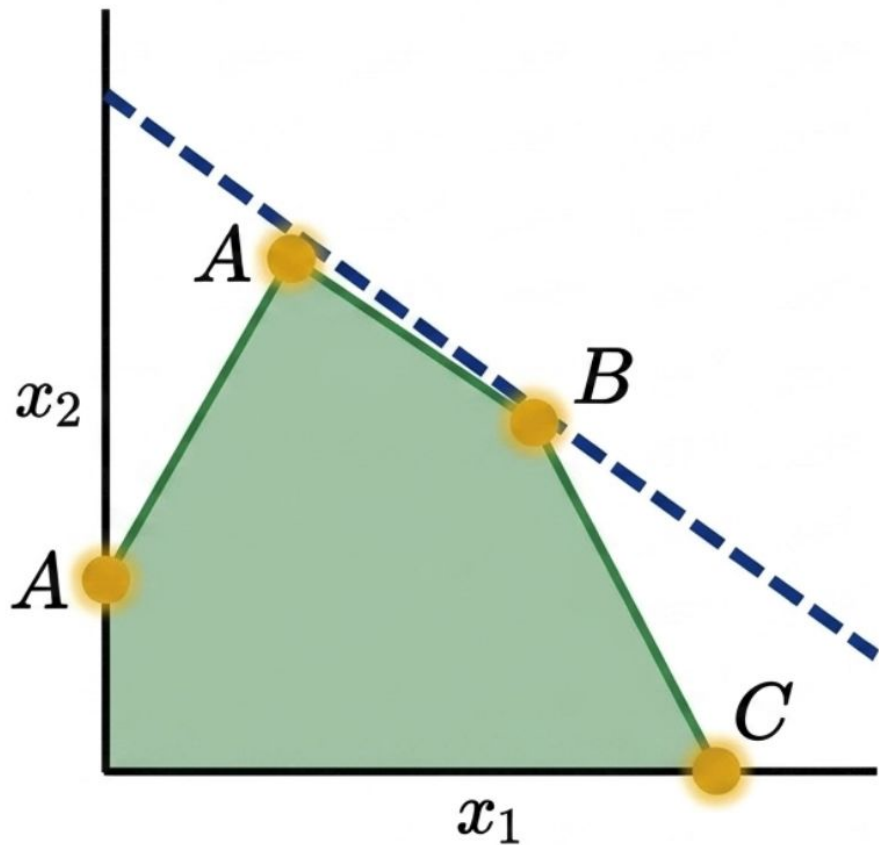
Half-Planes

Every constraint acts as a blade, splitting the world into 'allowed' and 'forbidden' zones.

The Intersection

The Feasible Region (Forest Green) is the exact intersection of all valid half-planes. Every possible real-world solution to the business problem lives somewhere inside this specific geometric space.

The Power of Extreme Points



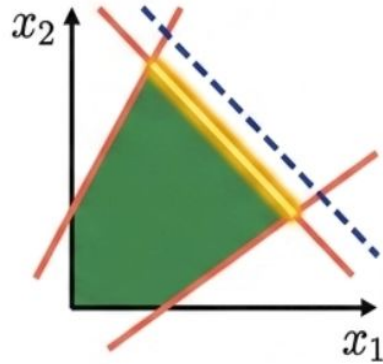
The Fundamental Theorem

If an optimal solution exists, it must occur at an extreme point (a vertex).

Why Corners?

The objective function is a straight line. As it pushes outward through the feasible region to maximize value, its final point of contact will always be a corner. We never need to check the infinite number of points inside the region.

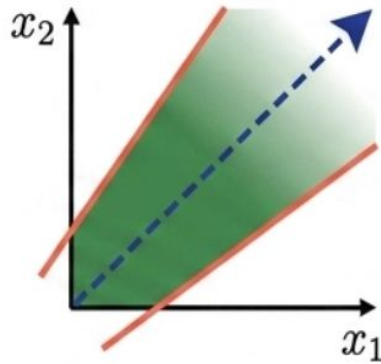
Diagnostic Matrix: When LP Goes Wrong



Multiple Optimal Solutions

Objective line is perfectly parallel to a constraint boundary.

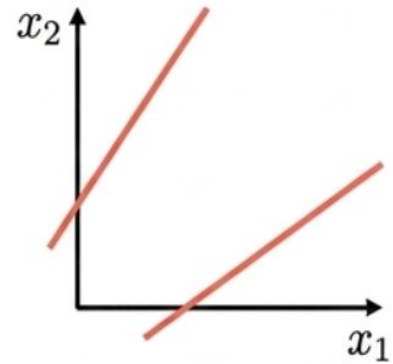
Management has alternative choices that yield the exact same peak profit.



Unbounded Solution

The feasible region is open-ended; the objective can grow to infinity.

A flawed model. It is impossible to achieve **infinite profit** with **finite** real-world resources.

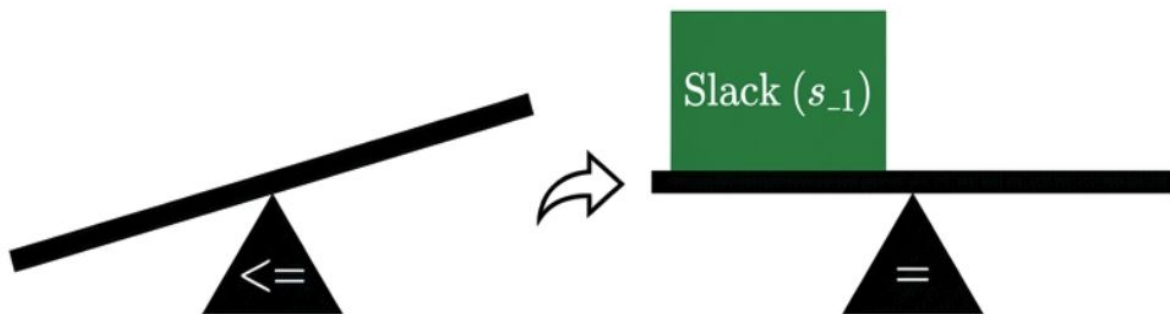


No Feasible Solution

Constraints contradict each other; no intersection exists.

The system is over-constrained. No **reality** exists that satisfies all requirements simultaneously.

Preparing the Algorithm: Standard Form



The Requirement

Algorithmic engines like Simplex cannot process inequalities. They require strict equations.

$\leq$ Constraints

Add a Slack Variable to the left side. It represents unused resources (e.g., leftover materials).

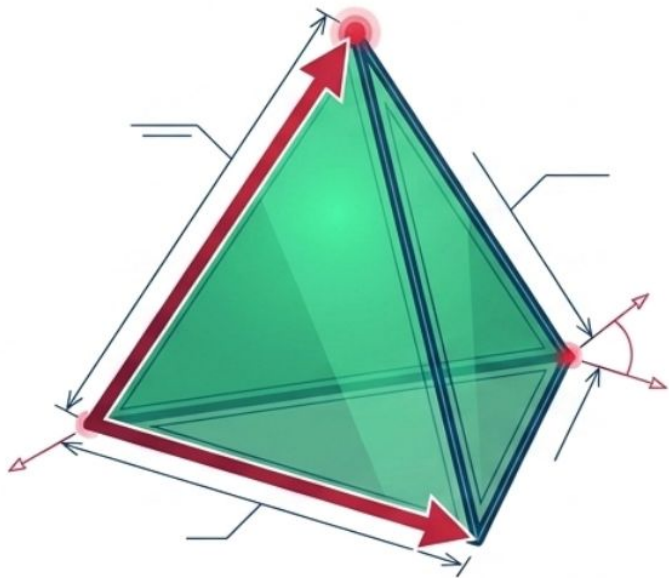
$\geq$ Constraints

Subtract a Surplus Variable from the left side. It represents excess production beyond minimum requirements.

Linear Optimization: Traversing the Feasible Region

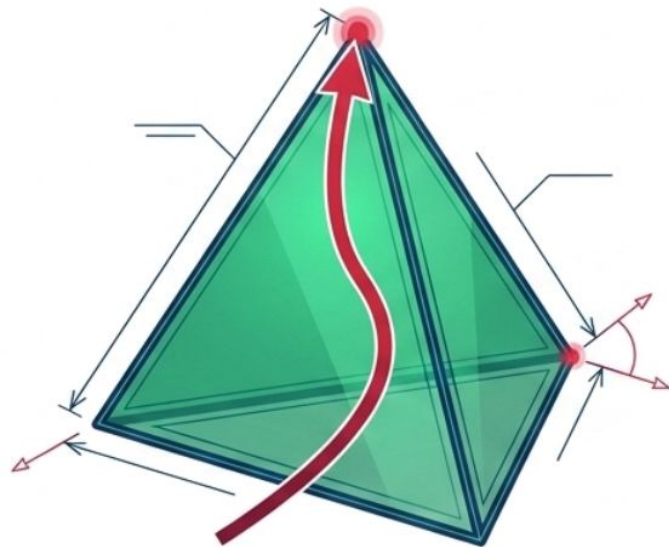
If an optimal LP solution exists, it will always occur at a corner point (or along an entire edge) of the feasible region. Any LP with only a single constraint is known as a knapsack problem.

The Simplex Algorithm (Dantzig, 1947)



Solves LPs by iteratively solving linear equations. It traverses the exterior edges of the feasible simplex, considering only the corners.

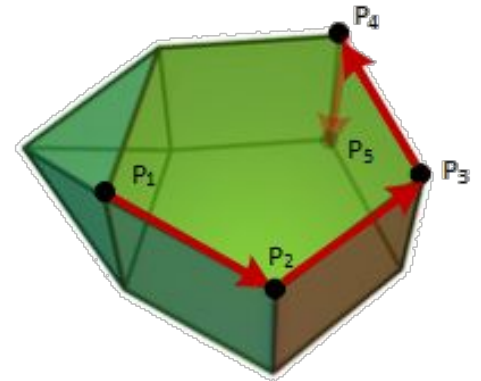
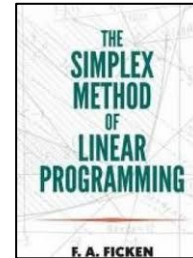
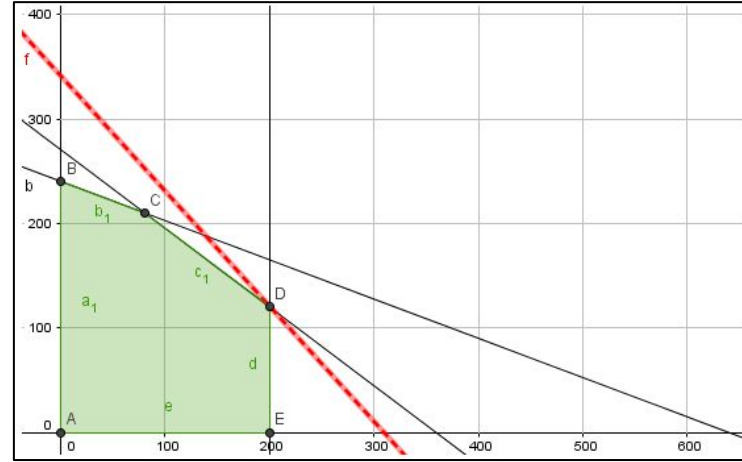
Interior Point Methods (Karmarkar, 1980s)



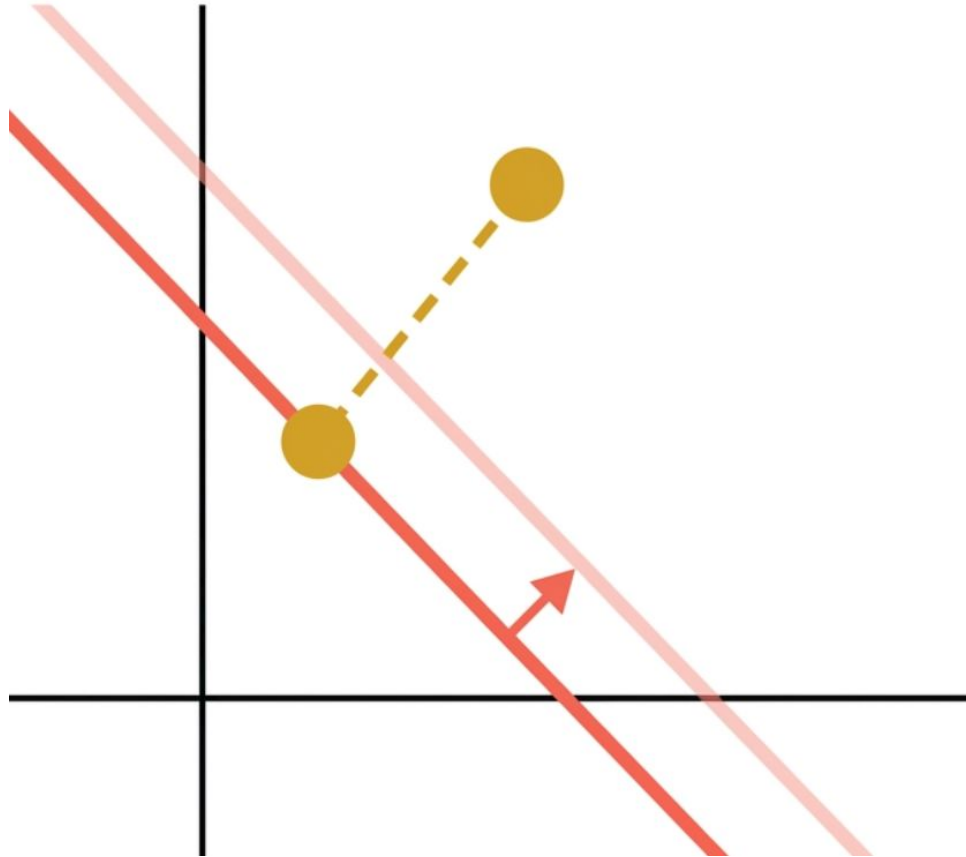
Reaches the optimal solution by cutting a path from inside the feasible set. Highly effective for a wider class of convex problems.

Some LP Concepts

- In a **linear programming** (LP) model, all the constraints and the objective are linear functions of the decision variables, which are not required to be integer numbers.
- The **Simplex algorithm** is typically used for solving LP models.
- All terms in the obj. function (and in each constraint) should have the same **dimensions** and **scale**.
- Any LP with only a single constraint is known as a **knapsack problem** (e.g., packing goods into a transport vehicle).
- If an optimal solution to an LP model exists, it will occur at a **corner** point of the feasible region.
- An LP can have **one** solution, **none**, or an **infinite** number of them (an entire simplex edge).



Economic Insights: The Shadow Price



Definition

The marginal economic worth of exactly one extra unit of a given resource. Found directly in the optimal Simplex tableau.

The Zero Rule

If a constraint is not binding (meaning there is slack, or unused resource), its shadow price is strictly zero.

Business Insight

You should never pay more for a resource than its shadow price.

Sensitivity Analysis: Operating Under Uncertainty

The Reality: Market costs and resource limits fluctuate constantly.
How resilient is our optimal solution?

Objective
Ranging
(Profit Margins)



How much can the market price of a product rise or fall before our current optimal corner is no longer the best choice?

Right-Hand Side
Ranging
(Resources)

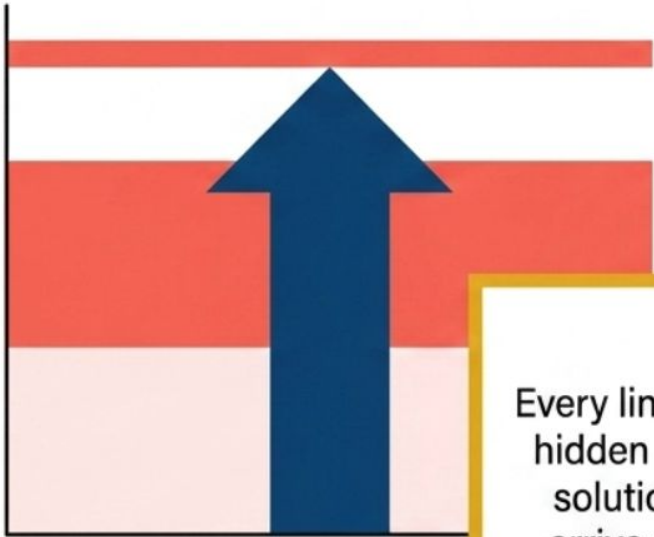


How much can a resource supply shrink or expand before the active constraints fundamentally change shape?

The Mirror World: Primal vs. Dual

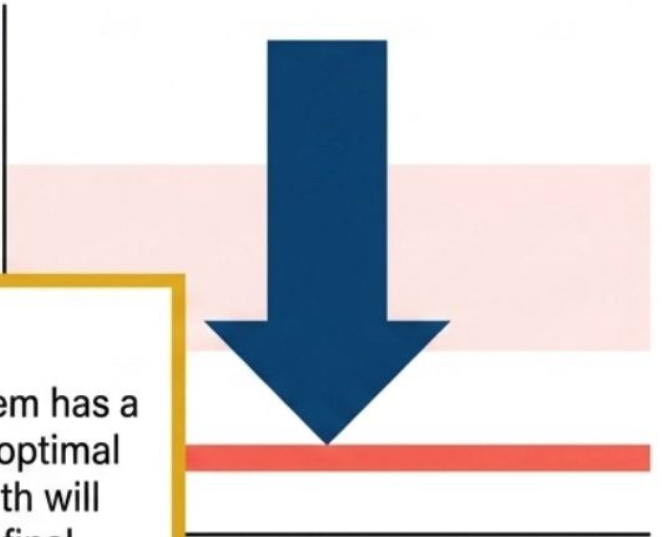
Primal (The Front)

Maximize Profit, subject to constraints on limited resources.



Dual (The Reflection)

Minimize the valuation of resources, subject to constraints on required profit margins.



The Magic

Every linear programming problem has a hidden alter-ego. If one has an optimal solution, so does the other. Both will arrive at the exact same same final optimal objective value (Z).

Optimization Duality: The Shadow Problem

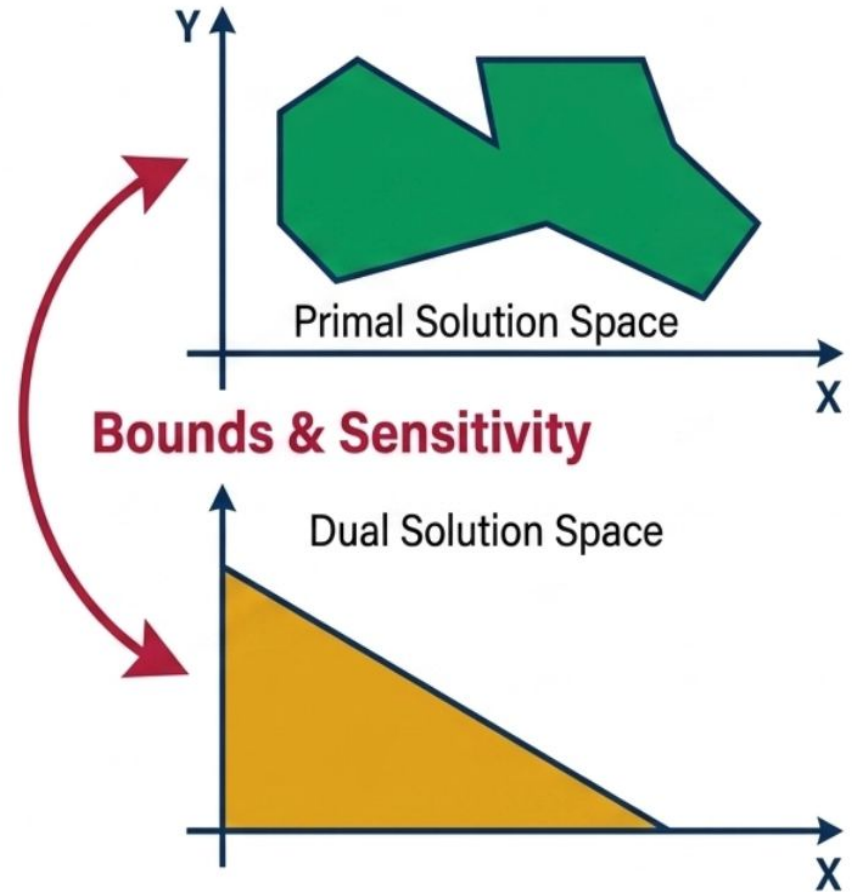
Every primal (P) optimization problem has a mirrored dual (D) counterpart.

The Mirror Rules

- If P is a MIN problem, D is a MAX problem (and vice versa).
- The number of variables in D equals the number of constraints in P.

Why it Matters

- A good solution to D computes a strict bound for the objective value in P.
- Often, the Dual has a superior mathematical or computational structure.
- Dual variables provide the keys to perform sensitivity analysis on the Primal.



Solver

Solver






> Objective (Minimize \$C\$25)

> Variables (C15:E15)

\$C\$19 >= \$F\$8

\$C\$20 >= \$F\$9

\$C\$21 >= \$F\$10

\$C\$23 <= \$G\$11

2

Add

Change

Delete

Model (Type: linear)

Engines (Standard LP/Simplex)

Note: better use the desktop version of Excel Solver instead of the online one.

Solving an LP with LINDO



LINDO

File Edit Solve Reports Window Help

<untitled>

```
min 0.25X1 + 0.10X2 + 0.08X3
st
0.70X1 + 0.80X2 + 0.00X3 >= 10    !calcium
0.90X1 + 0.80X2 + 0.80X3 >= 12    !iron
0.80X1 + 1.50X2 + 0.90X3 >= 15    !protein
0.50X1 + 0.60X2 + 0.40X3 <= 7.5    !flat
end
```

1

LINDO, LINGO, and What'sBest! are commercial software by Lindo Systems Inc. that allow to model and solve linear and nonlinear optimization problems. They are similar to other popular optimizers such as CPLEX, Gurobi or hexaly. A demo or even academic version is usually available.

2

LP OPTIMUM FOUND AT STEP 4

OBJECTIVE FUNCTION VALUE

1)	2.590000	
----	----------	--

VARIABLE	VALUE	REDUCED COST
X1	8.000000	0.000000
X2	5.500000	0.000000
X3	0.500000	0.000000

SLACK OR SURPLUS

2)	0.000000	-0.310000
3)	0.000000	-0.670000
4)	0.100000	0.000000
5)	0.000000	1.140000

NO. ITERATIONS= 4

2

Solving an LP with Pyomo (1/3)



```
In [1]: conda install -c conda-forge pyomo
```

```
In [4]: conda install -c conda-forge glpk
```

Pyomo is a Python-based open-source algebraic modeling language (similar to AMPL, AIMMS, or GAMS) that supports linear and nonlinear optimization modeling and solving. It can be connected to multiple solvers, among others **GLPK**, Gurobi, CPLEX, etc.

1

2

Build and save the **diet.dat** file with the input data for your model.

If it does not work, check with ChatGPT if newer versions of Pyomo use a different syntax to model inputs!

```
param: F:          c :=
      p1          0.25
      p2          0.10
      p3          0.08 ;

param: N:      Nmin  Nmax :=
      Calc    10.0   .
      Iron    12.0   .
      Prot    15.0   .
      Fat     .      7.5 ;

param a:
      Calc  Iron  Prot  Fat :=
      p1    0.7   0.9   0.8  0.5
      p2    0.8   0.8   1.5  0.6
      p3    0.0   0.8   0.9  0.4 ;
```

Solving an LP with Pyomo (2/3)



Built and save the model as diet.py

3

Notice that this is an **abstract model**, i.e., it can be used with different sets of data.

```
from pyomo.environ import *
infinity = float('inf')

model = AbstractModel()

# Foods
model.F = Set()
# Nutrients
model.N = Set()

# Cost of each food
model.c = Param(model.F, within=PositiveReals)
# Amount of nutrient in each food
model.a = Param(model.F, model.N, within=NonNegativeReals)
# Lower and upper bound on each nutrient
model.Nmin = Param(model.N, within=NonNegativeReals, default=0.0)
model.Nmax = Param(model.N, within=NonNegativeReals, default=infinity)

# Number of servings consumed of each food
model.x = Var(model.F, within=NonNegativeReals)

# Minimize the cost of food that is consumed
def cost_rule(model):
    return sum(model.c[i]*model.x[i] for i in model.F)
model.cost = Objective(rule=cost_rule)

# Limit nutrient consumption for each nutrient
def nutrient_rule(model, j):
    value = sum(model.a[i,j]*model.x[i] for i in model.F)
    return model.Nmin[j] <= value <= model.Nmax[j]
model.nutrient_limit = Constraint(model.N, rule=nutrient_rule)
```

Solving an LP with Pyomo (3/3)



```
In [12]: !pyomo solve --solver=glpk /home/aaajp/diet.py /home/aaajp/diet.dat
```

```
In [13]: !cat results.yml
```

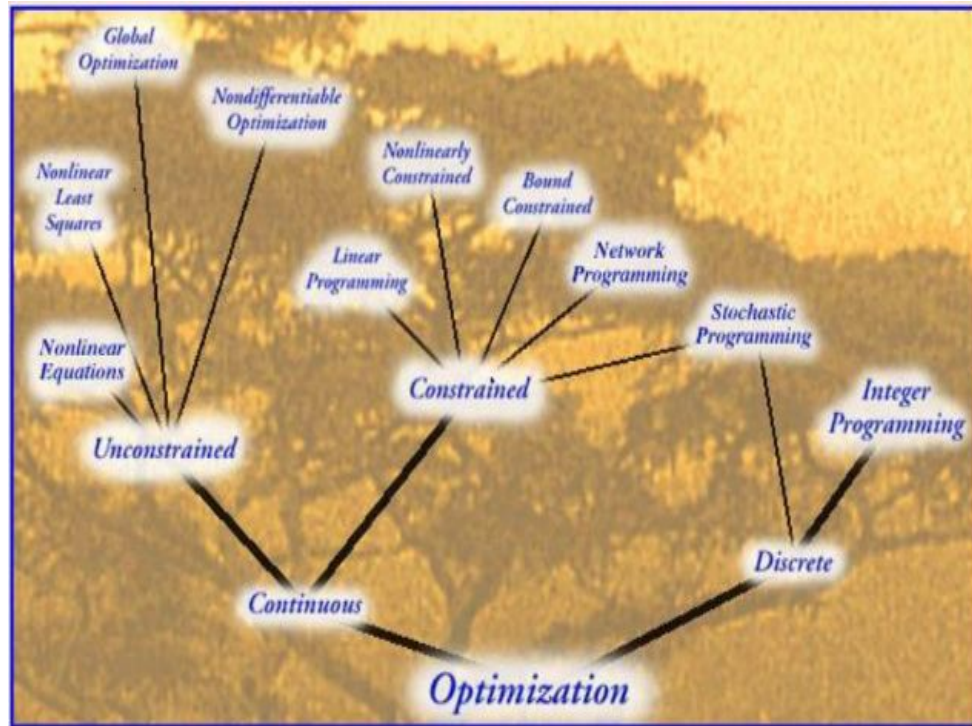
4

```
# -----  
#   Solver Information  
# -----  
Solver:  
- Status: ok  
  Termination condition: optimal  
  Statistics:  
    Branch and bound:  
      Number of bounded subproblems: 0  
      Number of created subproblems: 0  
  Error rc: 0  
  Time: 0.010757684707641602  
# -----  
#   Solution Information  
# -----  
Solution:  
- number of solutions: 1  
  number of solutions displayed: 1  
- Gap: 0.0  
  Status: feasible  
  Message: None  
  Objective:  
    cost:  
      Value: 2.58999999999999985  
  Variable:  
    x[p1]:  
      Value: 7.999999999999999  
    x[p2]:  
      Value: 5.5000000000000001  
    x[p3]:  
      Value: 0.5000000000000001  
  Constraint: No values
```

Make sure you use the right directories where your **diet.py** and **diet.dat** files are saved.

The NEOS Server for Optimization

- **NEOS** is a free internet-based service that lets you solve optimization problems using a wide range of solvers without needing to install them locally.
- On NEOS, you can submit problems in different formats (linear, nonlinear, integer, stochastic, etc.) through a web interface, email, or API. NEOS then forwards your problem to an appropriate solver (such as **CPLEX**, **Gurobi**, IPOPT, MINOS, and many others), runs it, and returns the results.
- It is widely used in research, teaching, and practice because it provides access to commercial and open-source solvers at no cost, though with **limits on problem size and runtime**.



Part III:

Solving Integer Programming Problems

The Integer Trap: Why Rounding Fails

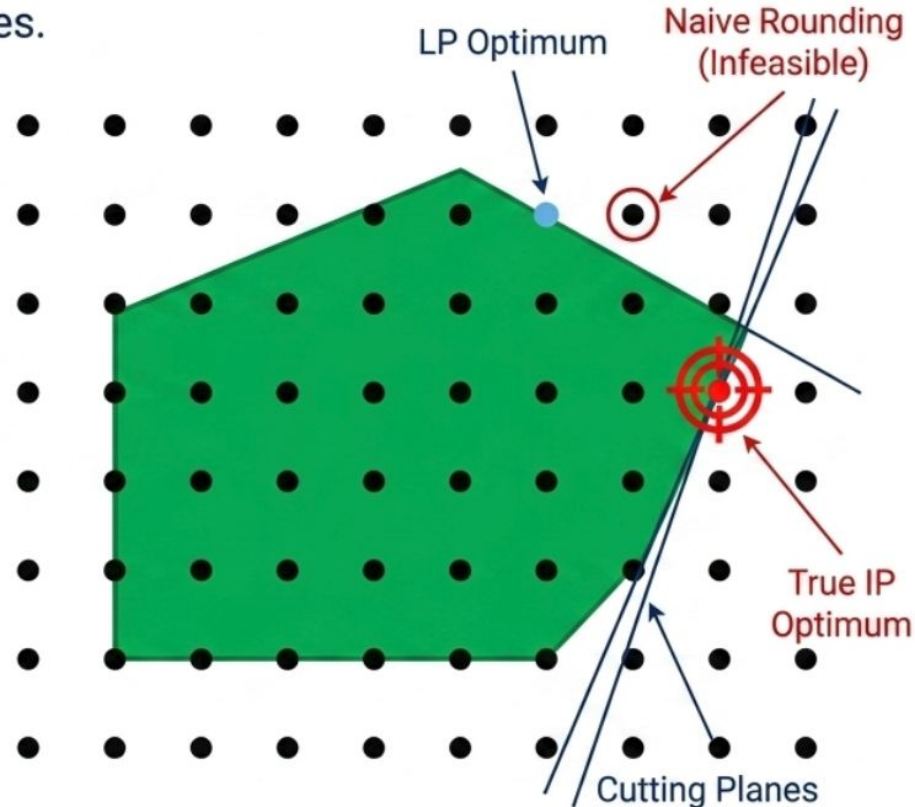
IP and MIP problems are drastically harder than LPs because variables are restricted to discrete choices.

The Danger of Rounding

Simply rounding an LP optimal solution often leads to an infeasible result outside the boundaries, or a deeply suboptimal one.

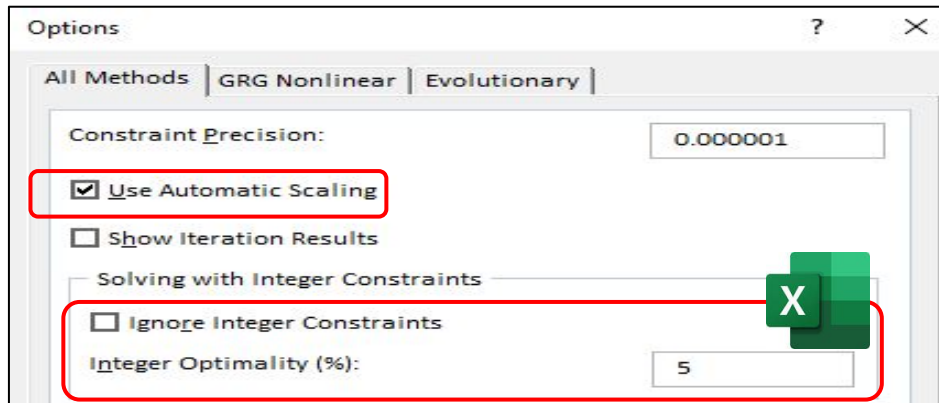
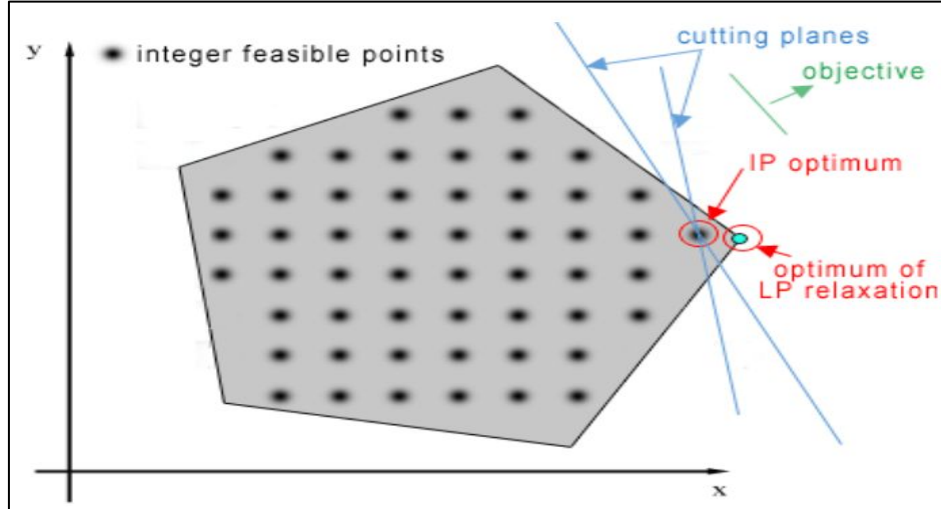
The Solution Toolkit

- IPs require implicit enumeration methods.
- ❑ **Branch-and-Bound:** Systematic tree search of possible integers.
 - ❑ **Branch-and-Cut:** Solving an LP version first, then using cutting planes (linear slices) to progressively reduce the choices of integer solutions.



Some IP Concepts

- Optimization models in which all / some of the variables must be integers are known as **integer / mixed int. progr. models (IP / MIP)**.
- IP / MIP problems are more difficult to solve than LP problems. They require implicit enumeration methods such as **branch and bound** or **branch and cut**.
- Solving an 'LP-relaxed' version (without integer constraints) will provide us with an **upper / lower bound** for the IP / MIP problem.
- Many **nonlinear problems** can be transformed into linear ones with the use of binary (0-1) variables.
- With a **tolerance setting** of $x\%$ the solver stops as soon as it finds a feasible integer solution that is within $x\%$ of the current upp / low bound to the optimal one.



Solving an IP with Open Solver (1/3)

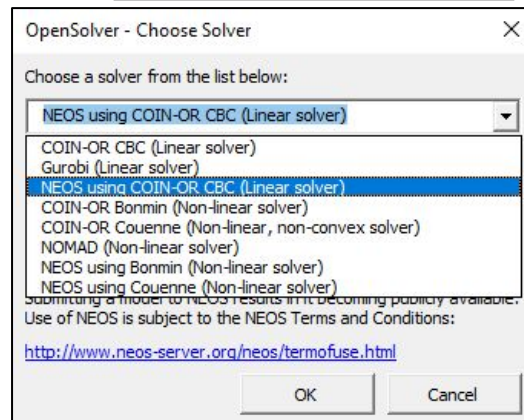
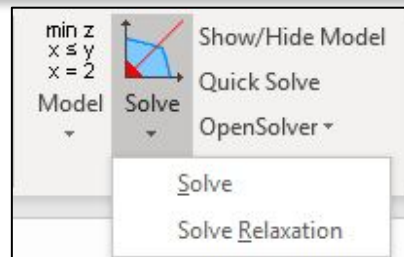


	A	B	C	D	E	F	G	H
1								
2		Goal: to set the equipment rent and units produced in order to maximize profit						
3		s.t.: effective capacity and available resources cannot be exceeded						
4								
5		Input data on products						
6			Shirts	Shorts	Pants	Skirts	Jackets	
7	Labor hours/unit		2.0	1.0	6.0	4.0	8.0	
8	Cloth/unit		3.0	2.5	4.0	4.5	5.5	
9								
10	Selling price/unit		35	40	65	70	110	
11	Variable cost/unit		20	10	25	30	35	
12	Fixed cost of renting equipment		1,500	1,200	1,600	1,500	1,600	
13								
14		Production plan, constraints on capacity						
15			Shirts	Shorts	Pants	Skirts	Jackets	
16	Rent equipment?		1	1	1	1	1	<- binary
17								
18	Units produced		90.0	150.0	10.0	15.0	80.0	
19			<=	<=	<=	<=	<=	
20	Effective capacity		1500.0	1800.0	666.7	1000.0	500.0	
21	note: effective_capacity = rent_equipment * MIN(labour_hours_avail / labor_hours_unit, cloth_avail / cloth_unit)							
22								
23		Constraints on resources						
24		Resource used			Available			
25	Labor hours	1,090.0	<=		4,000			
26	Cloth	1,192.5	<=		4,500			
27								
28		Monetary outputs						
29	Revenue	19,650						
30	Variable cost	6,800						
31	Fixed cost for equipment	7,400						
32	Profit	5,450	<- objective function					

Solving an IP with Open Solver (2/3)

Open Solver for Excel (advanced version) is a plug-in for Excel that extends the classical Solver capabilities. It can use locally installed COIN-OR solvers or it can be linked to a NEOS Server and use multiple solver engines.

2



OpenSolver - Model

What is AutoModel?

AutoModel

AutoModel is a feature of OpenSolver that tries to automatically determine the problem you are trying to optimise by observing the structure of the spreadsheet. It will turn its best guess into a Solver model, which you can then edit in this window.

Objective Cell:

Sheet1!\$C\$32

maximise

minimise

target value:

0

Variable Cells:

Sheet1!\$C\$16:\$G\$16,Sheet1!\$C\$18:\$G\$18

Constraints:

<Add new constraint>

\$C\$18:\$G\$18 <= \$C\$20:\$G\$20

\$C\$25:\$C\$26 <= \$E\$25:\$E\$26

\$C\$16:\$G\$16 bin

\$C\$16:\$G\$16

bin

Update constraint

Cancel

Delete selected constraint

☒ Make unconstrained variable cells non-negative
 ☒ Show named ranges in constraint list

Sensitivity Analysis

☐ List sensitivity analysis on the same sheet with top left cell:
 ☐ Output sensitivity analysis:

☒ updating any previous output sheet
 ☐ on a new sheet

Solver Engine:

Current Solver Engine: CBC

Solver Engine...

☒ Show model after saving

Clear Model

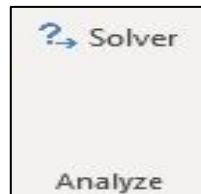
Options...

Save Model

Cancel

Solving an IP with Open Solver (3/3)

3



Production plan, constraints on capacity

	Shirts	Shorts	Pants	Skirts	Jackets
Rent equipment?	0	1	0	0	1 <- binary
Units produced	0.0	965.5	0.0	0.0	379.3
	<=	<=	<=	<=	<=
Effective capacity	0.0	1800.0	0.0	0.0	500.0
note: effective_capacity = rent_equipment * MIN(labour_hours_avail / labor_hours_unit, cloth_avail / cloth_unit)					

Constraints on resources

	Resource used		Available
Labor hours	4,000.0	<=	4,000
Cloth	4,500.0	<=	4,500

Monetary outputs

Revenue	80,345
Variable cost	22,931
Fixed cost for equipment	2,800
Profit	54,614 <- objective function

4

Note: up to a certain size, IP and nonlinear problems can also be solved with the classical Excel Solver. Despite this problem uses binary variables, it can still be solved with the linear solver engine.



Solver Parameters

Set Objective:

\$C\$32

To:

☒ Max
 ☐ Min
 ☐ Value Of:

0

By Changing Variable Cells:

\$C\$16:\$G\$16,\$C\$18:\$G\$18

Subject to the Constraints:

\$C\$16:\$G\$16 = binary

\$C\$18:\$G\$18 <= \$C\$20:\$G\$20

\$C\$25:\$C\$26 <= \$E\$25:\$E\$26

Add

Change

Delete

Reset All

Load/Save

☒ Make Unconstrained Variables Non-Negative

Select a Solving Method:

Simplex LP

Options

Solving Method

Select the GRG Nonlinear engine for Solver Problems that are smooth nonlinear. Select the LP Simplex engine for linear Solver Problems, and select the Evolutionary engine for Solver problems that are non-smooth.

Help

Solve

Close



Solving an IP with LINDO



<untitled>

```
MAX 15 X1 + 30 X2 + 40 X3 + 40 X4 + 75 X5
- 1500 Y1 - 1200 Y2 - 1600 Y3 - 1500 Y4 - 1600 Y5
```

SUBJECT TO

```
X1 - 1500 Y1 <= 0
X2 - 2250 Y2 <= 0
X3 - 666.6667 Y3 <= 0
X4 - 1000 Y4 <= 0
X5 - 500 Y5 <= 0
```

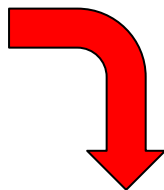
```
2 X1 + 1 X2 + 6 X3 + 4 X4 + 8 X5 <= 4000
3 X1 + 2.5 X2 + 4 X3 + 4.5 X4 + 5.5 X5 <= 4500
```

END

```
int Y1
int Y2
int Y3
int Y4
int Y5
```



Note: (a) in the objective function, variable cost per unit has been extracted from selling price for each product; and (b) in the first set of constraints, only the most restrictive of the two generated by the MIN(labor_cap, cloth_cap) has been kept.



OBJECTIVE FUNCTION VALUE

1) 54613.79

VARIABLE	VALUE	REDUCED COST
Y1	0.000000	1500.000000
Y2	1.000000	1200.000000
Y3	0.000000	1600.000000
Y4	0.000000	1500.000000
Y5	1.000000	1600.000000
X1	0.000000	22.241879
X2	965.517212	0.000000
X3	0.000000	14.827586
X4	0.000000	17.413794
X5	379.310333	0.000000

Trial Version Capacities:

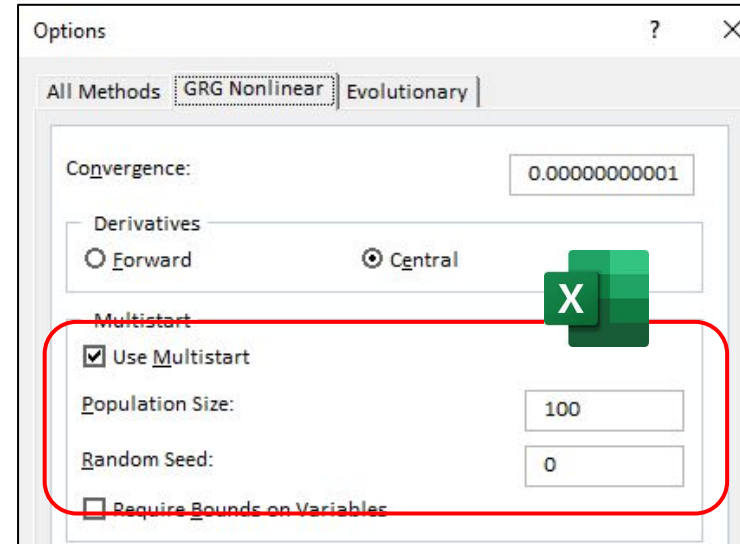
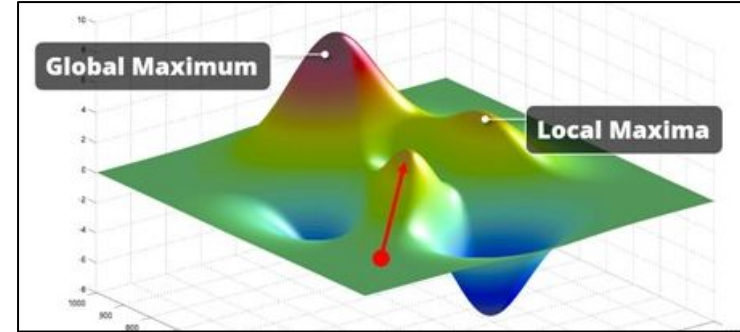
	Constraints	Variables	Integer Variables	Nonlinear Formulas	Global Variables
Classic LINDO	150	300	30	N/A	N/A
LINDO API	150	300	30	30	5
LINGO	150	300	30	30	5
What'sBest!	150	300	30	30	5

Part IV:

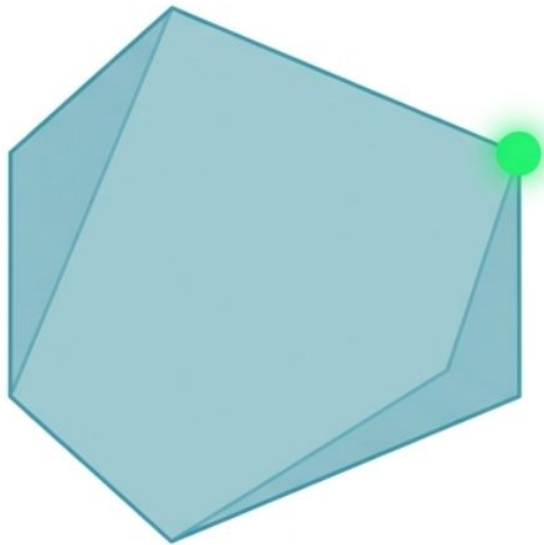
Solving Nonlinear Programming Problems

Some NLP Concepts

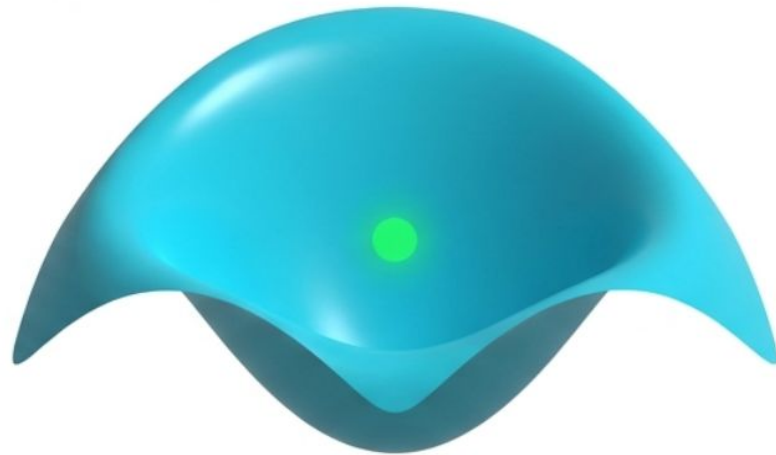
- Nonlinear Programming (NLP) problems have **nonlinear** objective functions and/or constraints.
- NLP models are more difficult to solve than linear ones, and **global optimality is not guaranteed** unless convexity requirements hold.
- The **real-world** often behaves in a nonlinear manner.
- In the **portfolio optimization problem**, the goal is to select a portfolio of investments that minimizes risk subject to meet an expected return. Risk is typically expressed as a nonlinear function.
- Excel Solver incorporates a **multistart** option, which provides an automatic way of starting from a number of starting solutions to search for global minima.



The Loss of the Extreme Point Guarantee



Linear Models. Solutions are mathematically guaranteed to exist at the extreme points (vertices) of the feasible region.



Nonlinear Models. Objective functions or constraints involve multiplicative interactions (xy), transcendental functions, or powers. The optimal solution can occur anywhere—including deep within the interior of the feasible region.

Real-World Arena: Portfolio Optimization

Balancing nonlinear risk with linear return constraints.

The Goal

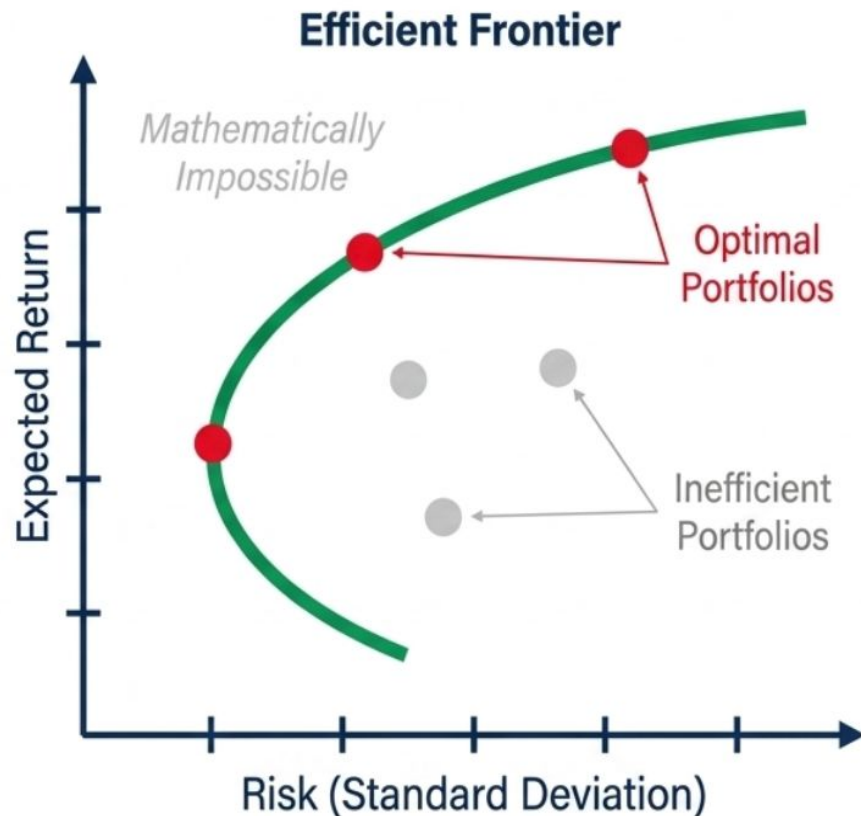
Select a portfolio that minimizes risk subject to meeting a minimum expected return.

The Math

- **Return (Linear):** Expected return is a linear sum of asset percentages.
- **Risk (Nonlinear):** Risk is expressed as a nonlinear variance function (calculated via covariance matrices from historical sample statistics).

Note on Practice

In complex non-linear models, tools like Excel Solver use multistart options—automatically launching from multiple starting solutions to search for the global minimum.

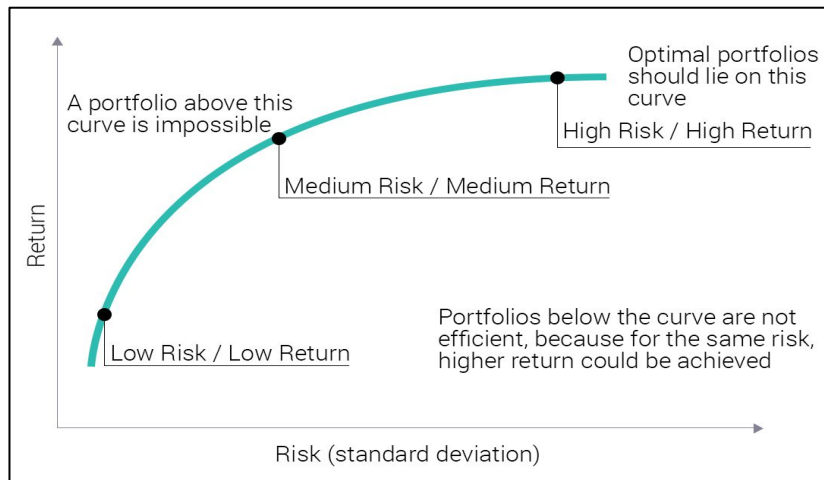


The Portfolio Optimization Problem

- Set of **assets** $\mathbf{A} = \{1, 2, \dots, n\}$.
- **R_i**: **annual return** earned on an euro invested in asset i in A . R_i is a random var with $E[R_i] = m_i$ and $StDev[R_i] = s_i$.
- **x_i**: % of budget invested in asset i in A , with $\text{Sum}\{x_i\} = 1$.
- **Expected annual return** of portfolio p : $E[R_p] = m_1 \cdot x_1 + m_2 \cdot x_2 + \dots + m_n \cdot x_n$.
- **Risk** of portfolio p : $\text{Var}[R_p] = \text{Sum}\{s_{ij} \cdot x_i \cdot x_j\}$, where $s_i = r_{ij} \cdot s_i \cdot s_j$ is the covariance between i and j (r_{ij} is the correlation between i and j).
- In practice, parameters m_i , s_i , and s_{ij} are **estimated** using sample statistics from historical data.
- Goal: to find the x_i that **minimize risk** while satisfying $E[R_p] \geq R$ (R is minimum return requested).

$$\begin{aligned} \min \quad & \sum_{i=1}^3 \sum_{j=1}^3 x_i x_j \sigma_{ij} \\ \text{s.t.} \quad & \sum_{i=1}^3 x_i \leq 1000.00, \\ & \sum_{i=1}^3 \bar{r}_i x_i \geq r_{\min}, \\ & x_i \geq 0 \quad i = 1, 2, 3. \end{aligned}$$

Model for 3 assets and a total budget of 1000 euros (in this case, x_i represents euros invested in asset i).



Solving an NLP with Solver (1/2)



CLASSICAL PORTFOLIO OPTIMIZATION PROBLEM

Decide the % of the total budget to invest in each asset (x_i)

Goal: to minimize risk while obtaining an expected return above a given threshold

Inputs based on instance PORT1 from the OR Library

Number of assets = 31
User threshold (R) = 0.0108003319

Exp. Return
0.0080565000

ERROR! <- Constraint 2

Min Sum($s(i,j) * x_i * x_j$) <- minimize Risk (\$N\$8)
s.t.:
(\$E\$8) Sum(x_i) = 1 <- use all budget
(\$F\$6) Sum($x_i * r_i$) >= R <- exp. return >= R
(\$E\$12:\$E\$42) x_i >= 0 <- percentage

Sum x_i = 1.000000 OK <- Constraint 1

Risk = 0.0026255561

avg. return $r_i = E[R(i)]$ stDev $s(i)$ investment x_i (in %) OK <- Constraint 3
 $x_i * r_i$

correlation covariance
Asset i Asset j $r(i,j)$ $s(i,j) = r(i,j) * s(i) * s(j)$
(a) $s(i,j) * x_i * x_j$ (b) $s(j,i) * x_j * x_i$
 $i \leq j$ $j > i$ (a)+(b)

Input data (port1.txt and portef1.txt files) available from:
<http://people.brunel.ac.uk/~mastijb/jeb/orlib/portinfo.html>

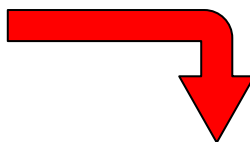
Asset #	avg. return $r_i = E[R(i)]$	stDev $s(i)$	investment x_i (in %)	$x_i * r_i$	Asset i	Asset j	$r(i,j)$	$s(i,j) = r(i,j) * s(i) * s(j)$	(a) $s(i,j) * x_i * x_j$ $i \leq j$	(b) $s(j,i) * x_j * x_i$ $j > i$	(a)+(b)
1	0.001309	0.043208	0.0000	0.000000	1	1	1.000000	0.001867	0.000000	0.000000	0.000000
2	0.004177	0.040258	0.0000	0.000000	1	2	0.562289	0.000978	0.000000	0.000000	0.000000
3	0.001487	0.041342	0.0000	0.000000	1	3	0.746125	0.001333	0.000000	0.000000	0.000000
4	0.004515	0.044396	0.0000	0.000000	1	4	0.707857	0.001373	0.000000	0.000000	0.000000
5	0.004515	0.044396	0.0000	0.000000	1	5	0.336386	0.001004	0.000000	0.000000	0.000000
6	0.004515	0.044396	0.0000	0.000000	1	6	0.704836	0.001635	0.000000	0.000000	0.000000
7	0.004515	0.044396	0.0000	0.000000	1	7	0.680165	0.001370	0.000000	0.000000	0.000000
8	0.004515	0.044396	0.0000	0.000000	1	8	0.629523	0.001237	0.000000	0.000000	0.000000
9	0.007115	0.053634	0.0000	0.000000	1	9	0.379162	0.000879	0.000000	0.000000	0.000000
10	0.003186	0.046923	0.0000	0.000000	1	10	0.724315	0.001469	0.000000	0.000000	0.000000
11	0.002093	0.047043	0.0000	0.000000	1	11	0.566894	0.001152	0.000000	0.000000	0.000000
12	0.005202	0.042955	0.2500	0.001301	1	12	0.638969	0.001186	0.000000	0.000000	0.000000
13	0.004489	0.039754	0.0000	0.000000	1	13	0.476764	0.000819	0.000000	0.000000	0.000000
14	0.003642	0.050224	0.0000	0.000000	1	14	0.632612	0.001373	0.000000	0.000000	0.000000
15	0.003960	0.036183	0.0000	0.000000	1	15	0.545067	0.000852	0.000000	0.000000	0.000000
16	0.000141	0.038844	0.0000	0.000000	1	16	0.427249	0.000717	0.000000	0.000000	0.000000
17	0.000282	0.043980	0.0000	0.000000	1	17	0.471693	0.000896	0.000000	0.000000	0.000000
18	0.000392	0.050467	0.0000	0.000000	1	18	0.537818	0.001173	0.000000	0.000000	0.000000
19	0.005294	0.058710	0.2500	0.001324	1	19	0.469064	0.001190	0.000000	0.000000	0.000000
20	0.004801	0.050000	0.0000	0.000000	1	20	0.659323	0.001424	0.000000	0.000000	0.000000
21	0.002699	0.044716	0.0000	0.000000	1	21	0.640393	0.001237	0.000000	0.000000	0.000000
22	0.001879	0.038128	0.0000	0.000000	1	22	0.537710	0.000886	0.000000	0.000000	0.000000
23	0.004656	0.045625	0.0000	0.000000	1	23	0.676022	0.001402	0.000000	0.000000	0.000000
24	0.003842	0.050449	0.0000	0.000000	1	24	0.694944	0.001402	0.000000	0.000000	0.000000
25	0.002690	0.053335	0.0000	0.000000	1	25	0.702592	0.001402	0.000000	0.000000	0.000000
26	0.004793	0.045450	0.0000	0.000000	1	26	0.308208	0.001402	0.000000	0.000000	0.000000
27	0.003286	0.047931	0.0000	0.000000	1	27	0.676855	0.001402	0.000000	0.000000	0.000000

Note: the objective function is nonlinear since it includes $x_i * x_j$ terms.



It is possible to use Python or VBA to iteratively call Solver using different levels of the requested return R , thus obtaining the points (risk, R) that define the efficient frontier.

For a return threshold $R = 0.0108003319$, Solver employs just a few seconds to find a feasible solution with Risk = **0.0046528245**. This risk value is virtually the same as the best-known solution (BKS) from the literature: 0.0046528244.



Solver Parameters

Set Objective:

To: ☐ Max ☒ Min ☐ Value Of:

By Changing Variable Cells:

Subject to the Constraints:

☒ Make Unconstrained Variables Non-Negative

Select a Solving Method:

Solving Method

Select the GRG Nonlinear engine for Solver Problems that are smooth nonlinear. Select the LP Simplex engine for linear Solver Problems, and select the Evolutionary engine for Solver problems that are non-smooth.

Buttons: Add, Change, Delete, Reset All, Load/Save, Options, Help, Solve, Close

31		Exp. Return			(F\$6) Sum{xi*ri} >= R	<- exp. return >= R	
0.0108003319	<=	0.0108003319 OK	<- Constraint 2		(E\$12:E\$42) xi >= 0	<- percentage	
Sum xi =	1.000000	OK	<- Constraint 1			Risk =	0.0046528245

Solving an NLP with LINGO (1/3)



```
Lingo Model - LINGO_PortfolioOptimizationTemplate

! Model to solve classical portfolio opt. problems (up to 300 variables using the LINGO demo version);
! Allows to consider multiple thresholds to generate the Unconstrained Efficient Frontier (UEF);
! Data is obtained from an open Excel file using Windows OLE and Excel range names;

SETS:
  ! Xi = quantity-to-invest in i-asset (in %), RETi = exp. return for i-asset;
  ASSET: X, RET;
  ! THRESHOLD = minimum expected return target requested by the user;
  THRESHOLD: TARGET;
  ! COVMAT = covariance matrix (triangular form);
  COVMAT(ASSET, ASSET) | &2 #GE# &1: COVARIANCE;
ENDSETS

! Model to compute Unconstrained Efficient Frontier (UEF);
SUBMODEL CLASSICALPO:
  ! Minimize the end-of-period variance in portfolio value: Min Sum(Xi * Xj * sigma(i, j));
  [RISK] MIN = @SUM( COVMAT(I, J) | I #LT# J: 2 * X(I) * X(J) * COVARIANCE(I, J)) +
              @SUM( COVMAT(I, J) | I #EQ# J: X(I) * X(J) * COVARIANCE(I, J));
  ! Use exactly 100% of the starting budget: Sum(Xi) = 1;
  [BUDGET] @SUM( ASSET: X) = 1;
  ! Required wealth at end of period: Sum(Xi * RETi) >= exp. return threshold;
  [RETURN] @SUM( ASSET: X * RET) >= R;
ENDSUBMODEL

DATA:
  ! Expected returns. Data is read from an open Excel file with a range name 'RET';
  RET = @OLE( );
  ! Covariance matrix, i.e., sigma(i, j). Data is read from an open Excel file with a range name 'COVARIANCE';
  COVARIANCE = @OLE( );
  ! Threshold for overall expected return. Data is read from an open Excel file with a range name 'TARGET';
  TARGET = @OLE( );
ENDDATA

CALC:
  ! Set output level to super terse;
  @SET('TERSEO', 2);
  @WRITE('      THRESHOLD      RISK', @NEWLINE(1));
  ! Loop over points of efficient frontier;
  @FOR( THRESHOLD(I):
    R = TARGET(I);      ! Update threshold value;
    @SOLVE(CLASSICALPO); ! Solve model with new threshold value;
    ! Write the objective value with 10 digits to the right of decimal point;
    @WRITE("      ", @FORMAT(R, "0.10f"), '      ', @FORMAT(RISK, "0.10f"), @NEWLINE(1));
  ); ! End @FOR loop;
ENDCALC
```



This is an abstract model, it reads the data from an open Excel file with the specified range names.

Solving an NLP with LINGO (2/3)



AutoSave Off

File Home Insert Page Layout Formulas Data Review View Help Search

Clipboard Font Alignment Number

Normal Check Ce

Conditional Formatting Table

TARGET 0.010865

INPUTS FOR LINGO MODEL - CLASSICAL PORTFOLIO OPTIMIZATION PROBLEM with MULTIPLE THRESHOLDS (UEF)

DATA ORIGIN FILE: PORT1.TXT

Asset #	avg. return $r_i = E[R(i)]$	stDev $s(i)$	Asset i	Asset j	correlation $r(i,j)$	covariance $s(i,j) = r(i,j)*s(i)*s(j)$	Threshold (R)
1	0.001309	0.043208	1	1	1.000000	0.001867	0.0108650000
2	0.004177	0.040258	1	2	0.562289	0.000978	0.0108609579
3	0.001487	0.041342	1	3	0.746125	0.001333	0.0108569167
4	0.004515	0.044896	1	4	0.707857	0.001373	0.0108528746
5	0.010865	0.069105	1	5	0.336386	0.001004	0.0108488325
6	0.001759	0.053671	1	6	0.704836	0.001635	0.0108447913
7	0.002594	0.046613	1	7	0.680165	0.001370	0.0108407492
8	0.004950	0.045492	1	8	0.629523	0.001237	0.0108367071
9	0.007115	0.053634	1	9	0.379162	0.000879	0.0108326659
10	0.003186	0.046923	1	10	0.724315	0.001469	0.0108286238
11	0.002093	0.047043	1	11	0.566894	0.001152	0.0108245817
12	0.005202	0.042955	1	12	0.638969	0.001186	0.0108205405
13	0.004489	0.039754	1	13	0.476764	0.000819	0.0108164984
14	0.003642	0.050224	1	14	0.632612	0.001373	0.0108124572
15	0.003960	0.036183	1	15	0.545067	0.000852	0.0108084151
16	0.000141	0.038844	1	16	0.427249	0.000717	0.0108043730
17	0.000282	0.043980	1	17	0.471693	0.000896	0.0108003319
18	0.000392	0.050467	1	18	0.537818	0.001173	0.0107962898
19	0.005294	0.058710	1	19	0.469064	0.001190	0.0107922476
20	0.004801	0.050000	1	20	0.659323	0.001424	0.0107882065
21	0.002699	0.044716	1	21	0.640393	0.001237	0.0107841644
22	0.001879	0.038128	1	22	0.537710	0.000886	0.0107801222
23	0.004656	0.045625	1	23	0.676022	0.001333	0.0107760811

Solving an NLP with LINGO (3/3)



Solution Report - LINGO_PortfolioOptimizati...

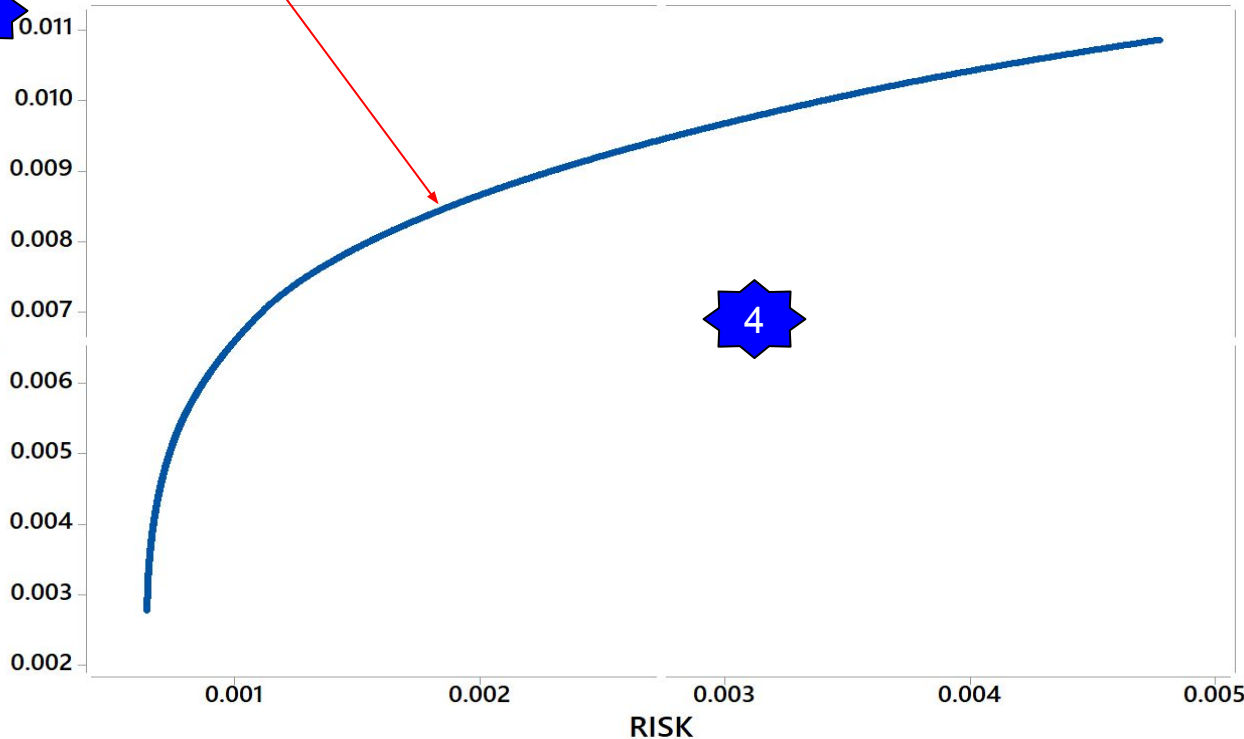
THRESHOLD	RISK
0.0108650000	0.0047755003
0.0108609579	0.0047677411
0.0108569167	0.0047599921
0.0108528746	0.0047522574
0.0108488325	0.0047445352
0.0108447913	0.0047368236
0.0108407492	0.0047291261
0.0108367071	0.0047214399
0.0108326659	0.0047137676
0.0108286238	0.0047061060
0.0108245817	0.0046984567
0.0108205405	0.0046908212
0.0108164984	0.0046831961
0.0108124572	0.0046755852
0.0108084151	0.0046679849
0.0108043730	0.0046603970
0.0108003319	0.0046528232
0.0107962898	0.0046452600
0.0107922476	0.0046377088
0.0107882065	0.0046301721
0.0107841644	0.0046226458
0.0107801222	0.0046151315
0.0107760811	0.0046076317
0.0107720390	0.0046001424
0.0107679968	0.0045926653
0.0107639557	0.0045852025
0.0107599136	0.0045777503
0.0107558715	0.0045703104
0.0107518303	0.0045628846
0.0107477882	0.0045554694
0.0107437461	0.0045480665
0.0107397049	0.0045406777
0.0107356628	0.0045332996
0.0107316216	0.0045259355
0.0107275795	0.0045185820
0.0107235374	0.0045112409

It takes LINGO less than a minute to compute all the (risk, return) points from which an efficient frontier can be built.



0.011

RETURN (threshold)

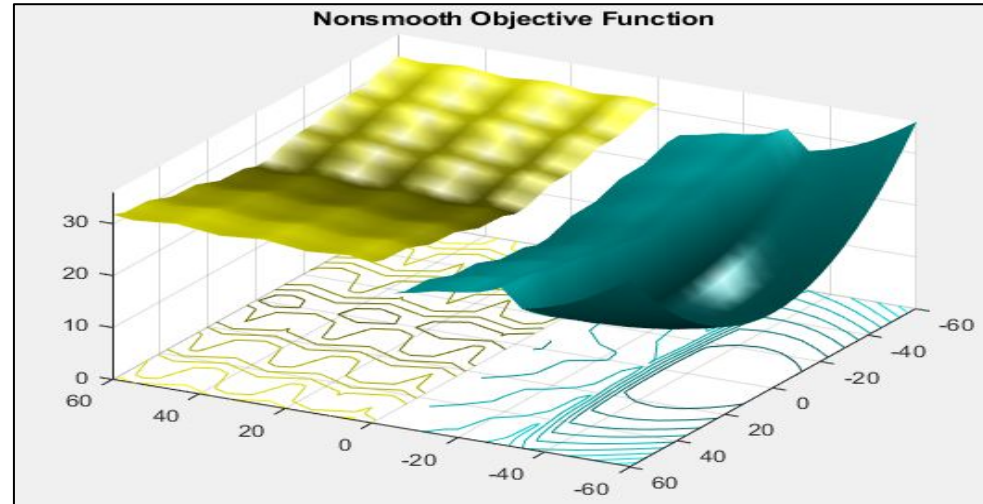
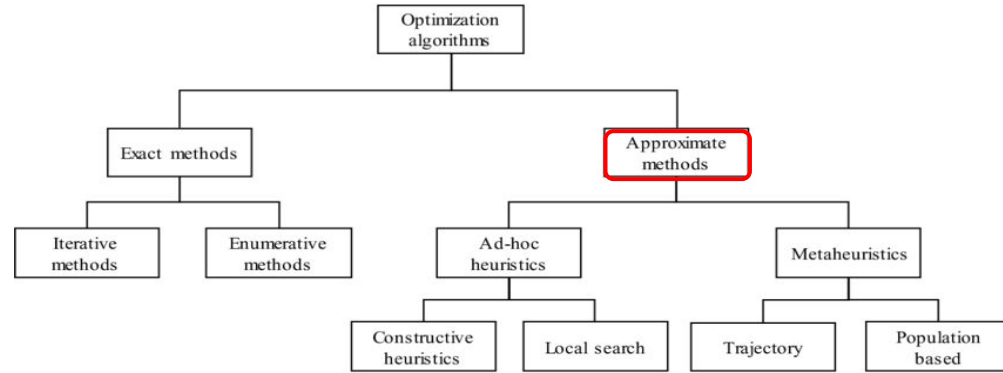


Part V:

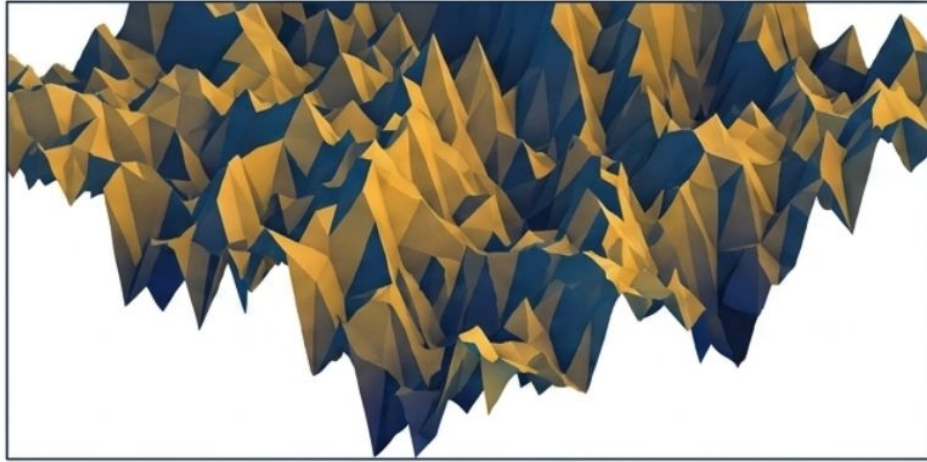
Solving Nonsmooth Problems

Some Nonsmooth Concepts

- In problems where exact methods do not perform well, approximate methods such as **genetic algorithms (GAs)** can be used.
- Models including functions like **IF**, **MAX**, **MIN**, or **ABS** (e.g., models with soft constraints that generate penalty costs) often contain **nonsmooth** objective functions. There are some strategies to “linearize” these models, but this is not always possible or easy.
- Also, many IP / MIP problems are **NP-hard**, i.e., approximate methods are needed in large-sized instances.
- GAs performance is not good on problems with many constraints. One strategy is to include a **constraint-violation penalty** as part of the obj. function.



Tackling the Nonsmooth and NP-Hard

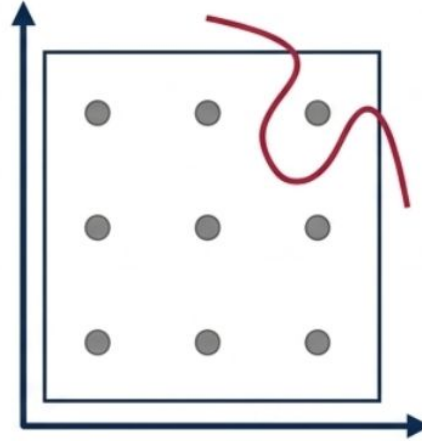


The Nonsmooth Terrain

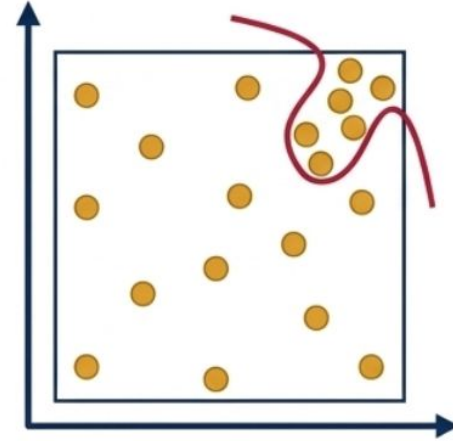
The Problem

Models containing logic functions (IF, MAX, MIN, ABS, or soft constraint penalties) create nonsmooth objective functions. Derivatives fail entirely.

Many IP/MIP problems at scale become NP-hard.



Failing Grid Search



Successful Random Search

The Approximate Toolkit

- Grid Search often misses irregular optima entirely.
- Randomized Search (Simulated Annealing, Tabu Search) introduces controlled randomness to explore jagged terrain.
- Genetic Algorithms (GAs) mimic evolution, though they struggle with heavy constraints unless penalty costs are added to the objective function.

The Optimization Playbook: Synthesis

A diagnostic guide to matching the problem architecture to the algorithmic toolkit.

If the problem is...	Characterized by...	Use this Toolkit...
Linear & Continuous	Straight lines, unrounded variables	Simplex or Interior Point
Non-Linear Continuous	Smooth curves, derivatives $\neq 0$	Lagrangian & KKT Conditions
Discrete / Indivisible	Integer requirements	Branch-and-Bound, Cutting Planes
Time-Dependent	Sequential decision states	Dynamic Programming (Backward Induction)
Nonsmooth / NP-Hard	Logic gates (IF/MAX), jagged terrain	Heuristics, Random Search, Genetic Algorithms

Always confirm convexity. Convexity ensures the peak you find is the only peak that matters.

Solving a Nonsmooth (1/2)



	A	B	C	D	E	F	G	H	I
1									
2		Storing products in truck compartments to minimize shortage (customers without service)							
3		If demand is not meet, a shortage cost per unit is applied.							
4		Also, if a maximum shortage level is exceeded, a penalty cost per extra unit is applied.							
5									
6		Unit shortage cost (for not meeting demand) and unit penalty cost (for exceeding the max. shortage constraint)							
7		Product	Cost / liter						
8		1	10.00 Eur						1
9		2	8.00 Eur						
10		3	6.00 Eur						
11									
12		Shortage penalty	100.00 Eur						
13									
14		Storing decisions							
15		Compartment	Product	Amount		Capacity			
16		1	1	2,700.0	<=	2,700.0			
17		2	2	2,800.0	<=	2,800.0			
18		3	3	1,100.0	<=	1,100.0			
19		4	1	1,800.0	<=	1,800.0			
20		5	2	3,400.0	<=	3,400.0			
21									
22		Shortages							
23		Product	Amount Stored	Demand	Shortage	Max Shortage	Shortage Violation		
24		1	4,500.0	2,900	0.0	900.0	0.0		
25		2	6,200.0	4,000	0.0	900.0	0.0		
26		3	1,100.0	4,900	3,800.0	900.0	2,900.0		
27									
28		Costs and penalties							
29		Shortage cost	22,800.00						
30		Penalty cost	290,000.00						
31		Total cost	312,800.00 Eur						
32									

Note: this model contains cells with MAX(), SUMIF(), etc., which account for nonsmooth penalty costs when soft constraints are violated.

Solving a Nonsmooth (2/2)



Solver Parameters

Set Objective:

To: ☐ Max ☒ Min ☐ Value Of:

By Changing Variable Cells:

Subject to the Constraints:

\$C\$16:\$C\$20 <= 3

\$C\$16:\$C\$20 = integer

\$C\$16:\$C\$20 >= 1

\$D\$16:\$D\$20 <= \$F\$16:\$F\$20

\$D\$16:\$D\$20 >= 0

Add

Change

Delete

Reset All

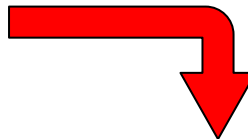
Load/Save

☐ Make Unconstrained Variables Non-Negative

Select a Solving Method: Options

Solving Method

Select the GRG Nonlinear engine for Solver Problems that are smooth nonlinear. Select the LP Simplex engine for linear Solver Problems, and select the Evolutionary engine for Solver problems that are non-smooth.



Storing decisions		
Compartment	Product	Amount
1	2	2,700.0
2	1	2,800.0
3	2	1,100.0
4	3	1,703.3
5	3	3,223.6
Shortages		
Product	Amount Stored	Demand
1	2,800.0	2,900
2	3,800.0	4,000
3	4,926.9	4,900
Costs and penalties		
Shortage cost	2,600.00	
Penalty cost	0.00	
Total cost	2,600.00	Eur

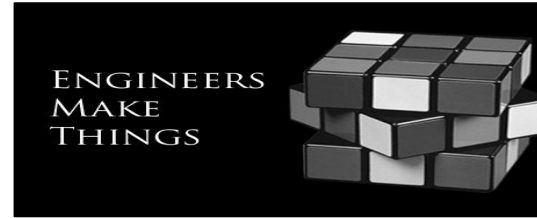
Note: depending on the starting solution, the Solver might reach a near-optimal solution quite quickly or need several minutes to do it.

Part VI:

Conclusions and Activities

Conclusions

- **Optimization** approaches allow us to design efficient systems and processes. Each opt. problem needs to be solved with the proper methods, either **exact** or **approximate**.
- **Linear programming** (LP) and **integer programming** (IP) models contain linear obj. functions and constraints. **Nonlinear programming** (NLP) models require different algorithms. The same is true for **nonsmooth opt.** models, which might require the use of approximate methods.
- A plethora of excellent **optimization software** are available. The one to use depends on the problem at hand as well as on a personal choice.
- Modeling skills and **analytical thinking** are probably the most relevant assets for applying optimization in the real world.



Industrial
Engineers
Make Things
Better



https://www.youtube.com/watch?v=sFWrmpXPVJw&feature=emb_logo



Lab / Homework Activities

1. Based on the references below, prepare a 3-page tutorial article in PDF introducing the basic version of the portfolio optimization problem, explaining its quadratic programming model, and solving the benchmark instances port1 to port5 with a solver like Gurobi, Cplex, OSQP, HiGHS or SCIP (explain your choice). Provide computational times and a graphical representation of both your unconstrained efficient frontier and the one generated when using the solutions in portf1 to portf5. Include Python / Julia code in a separate file.

References:

- Beasley, J. E. (2013). Portfolio optimisation: models and solution approaches. In Theory Driven by Influential Applications (pp. 201-221). Informs.

<https://people.brunel.ac.uk/~mastjjb/jeb/orlib/portinfo.html>

$$\begin{aligned} \min_{x_1, \dots, x_n} \quad & \sum_{i=1}^n \sum_{j=1}^n \sigma_{ij} x_i x_j \\ \text{s.t.} \quad & \sum_{i=1}^n \mu_i x_i \geq R_{\min}, \\ & \sum_{i=1}^n x_i = 1, \\ & x_i \geq 0, \quad i = 1, \dots, n. \end{aligned}$$

